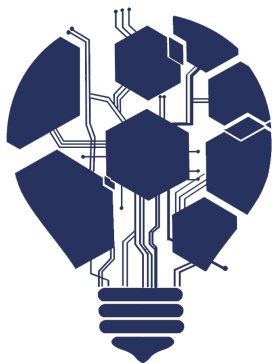


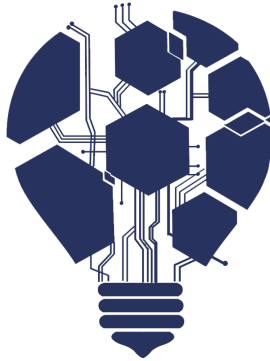
Cryptanalysis of ARX-based White-box Implementations : Section 6.4 ~ 6.5

26.09.01

류지은



- ① 1: Locating Linear Bits
- ② 2: Triangularization of the outer affine maps
- ③ 3: Recovery of the outer affine maps (left branches)
- ④ 4: Recovery of the Feistel affine maps (right to left XOR and addition)
- ⑤ 5: Correction of the first carry bit



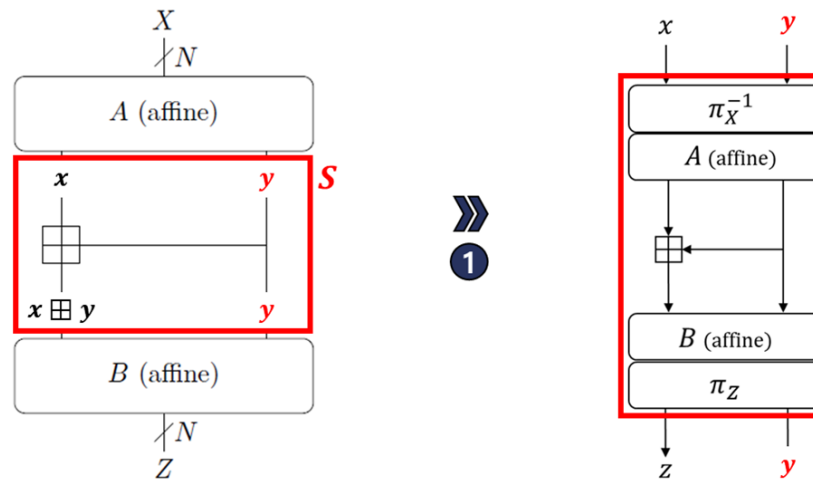
- ① 1: Locating Linear Bits
- ② 2: Triangularization of the outer affine maps
- ③ 3: Recovery of the outer affine maps (left branches)
- ④ 4: Recovery of the Feistel affine maps (right to left XOR and addition)
- ⑤ 5: Correction of the first carry bit

Locating Linear Bits

먼저, 라운드 함수의 encoding에서 선형 부분공간을 구별함으로써
라운드 함수 구조를 단순화하고 추후 수행될 공격이 적용될 범위를 축소한다.

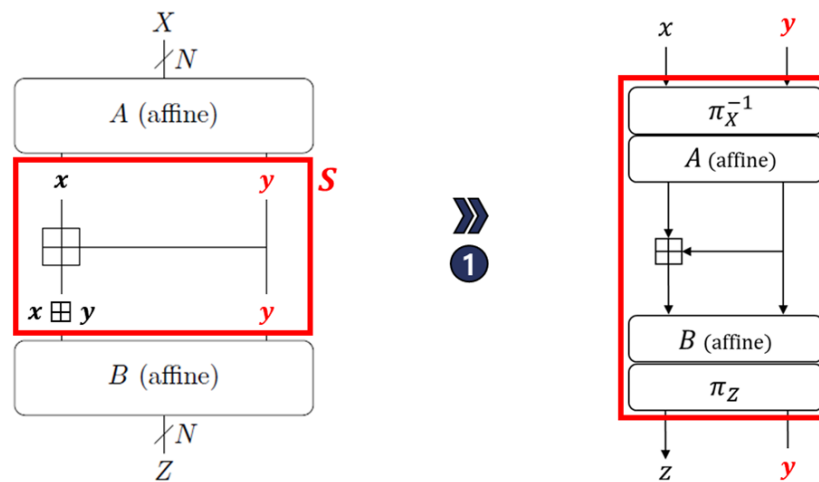
- ARX round를 S -layer에 affine encoding (A, B)를 추가한 형태로 볼 때,
encoding을 제외한 라운드 함수가 $S(x||y) = (x \boxplus y||y)$ 라는 성질을 활용한다.
- $\textcircled{1} = B \circ S \circ A$ 의 A, B 는 affine map이므로 S 의 basis transform만 수행하는 것으로 볼 수 있다.

⇒ Right branch에 대한 $n + 1$ 개 선형 관계를 찾아 basis transformation을 통해 분석하기 쉬운 구조로 전환한다.



Locating Linear Bits

해당 단계의 목표는 라운드 함수의 right branch를 고정하는 것이다.



라운드 함수의 S layer인 $S(x, y) = (x \boxplus y, y)$ 는

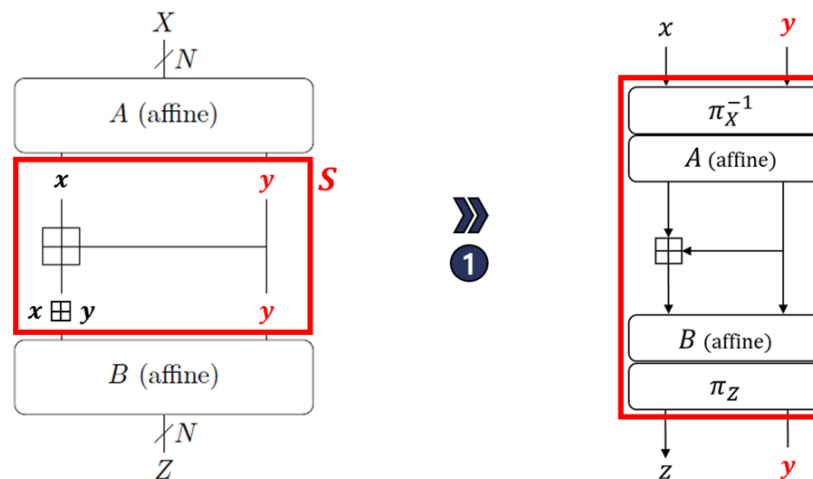
$$x = (x_{n-1}, \dots, x_0), \quad y = (y_{n-1}, \dots, y_0), \quad z = (z_{2n-1}, \dots, z_n, z_{n-1}, \dots, z_0)$$

에 대하여

$$\begin{cases} z_i = y_i & \text{for } i = 0, \dots, n-1 \\ z_n = x_0 \oplus y_0 \end{cases}$$

이라는 $n + 1$ 개의 affine relation을 가진다.

Locating Linear Bits



S layer에 합성된 A, B 는 affine map이므로 $n + 1$ 개 선형 관계가 이루는 부분 선형 공간의 차원 수를 보존한다.

이제 $n + 1$ 개의 선형 관계를 가지는 라운드 함수 $\tilde{S} = B \circ S \circ A$ 에 대하여 이 선형 관계를 보존하면서 right branch를 고정하는 새로운 affine map π_x, π_z 를 구하는 방법을 생각한다.

라운드 함수의 입력 $X = x \| y$ 에 대하여 $Z = z \| y$ 는 π_x, π_z 에 대하여

$$\begin{cases} \pi_x(y) = \pi_z(y) + c_y & (c_y: \text{affine map 보정}) \\ \pi_x(x_0) = \pi_z(z_0) + c_y \end{cases}$$

를 만족할 때, right branch가 고정된 것이다.

Locating Linear Bits

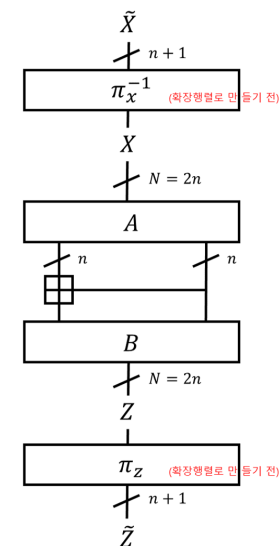
S 가 $n + 1$ 개의 선형관계를 가지므로,

$$\pi_x(X) = \pi_z(Z) + c_y$$

를 만족하는 선형 관계 역시 $n + 1$ 개이고 이를 right branch로 몰아서 right branch의 값이 고정되게 만들 수 있다.

$$\pi_x(X) = \pi_z(Z) + c_y \implies \pi_x(X) + \pi_z(Z) + c_y = \begin{pmatrix} X & Z & 1 \end{pmatrix} \begin{pmatrix} \pi_x \\ \pi_z \\ c_y \end{pmatrix} = 0$$

이므로 임의의 (X, Z) 에 대하여 상기 식이 성립하는 π_x, π_z 는 $N + \epsilon$ 개 이상의 (X, Z) 를 샘플링하고 이에 대한 커널을 구하여 생성할 수 있다.



S, A, B 의 특성은 $B \circ S \circ A$ 의 입출력으로 구성된 공간 V 가 $n + 1$ 개의 affine relation을 가지도록 $\dim \ker V = n + 1$ 을 보장한다. 따라서 kernel V 를 구하면 $\tilde{S}$ 의 $n + 1$ 개의 선형 관계를 찾을 수 있으며,

$$B^T = \text{basis}(\ker V)^T = \begin{pmatrix} \pi_x & \pi_z & c_y \end{pmatrix}$$

는 $\tilde{S}$ 내에서 섞인 $n + 1$ 개의 선형 관계를 걸러내는 역할을 한다.

π_x 는 affine map으로 $n + 1$ 개 독립 변수에 대한 $n + 1$ 개 일차항 선형 결합을 보존하며, π_z 역시 $n + 1$ 개의 출력에 대하여 일차항 선형 결합으로 구성됨을 보장한다.

출력이 일차항 입력 $n + 1$ 개와 선형 관계를 가지도록 π_z 를 구함

π_z 가 걸러내는 $n + 1$ 차원 부분 공간은 S 의 $n + 1$ 비트 right branch가 B 에 의해 변환된 좌표 축을 교정한다.

Locating Linear Bits

논문의 Algorithm 1은 이 과정을 절차화 한 것이다.

Algorithm 1 Recovering input/output matching affine subspaces

Input: oracle $\mathbb{O}$ implementing bijective modular addition with affine encodings

Output: affine maps π_x, π_z such that $\mathbb{O}' = \pi_z \circ \mathbb{O} \circ \pi_x^{-1}$ is such that $\mathbb{O}'(x||y) = (z||y)$ for all $(x, y) \in \mathbb{F}_2^{n-1} \times \mathbb{F}_2^{n+1}$, with $z \in \mathbb{F}_2^{n-1}$

```

1: for  $i \in \{0, \dots, N + \epsilon - 1\}$  do
2:    $x^{(i)} \xleftarrow{\$} \mathbb{F}_2^N$ 
3:    $y^{(i)} \leftarrow \mathbb{O}(x^{(i)})$ 
4:    $v^{(i)} \leftarrow (x^{(i)} || y^{(i)} || 1)$ 
5: end for
6:  $V \leftarrow$  matrix with rows  $\{v^{(i)}\}_i$ 
7:  $B \leftarrow$  basis(ker  $V$ ) so that  $V \times B = 0$ 
8:  $(\pi_x, \pi_z, c_y) \leftarrow B^T$ , where  $\pi_x, \pi_z \in \mathbb{F}_2^{(n+1) \times N}$ ,  $c_y \in \mathbb{F}_2^{(n+1) \times 1}$ 
9: return  $\pi_x, \pi_{\oplus}(0^{n-1} || c_y^T)$ 

```

위 알고리즘은 X 를 $x^{(i)}$, Z 를 $y^{(i)}$ 로 표기하며, X, Z 의 하위 $n + 1$ 개 비트에 대하여

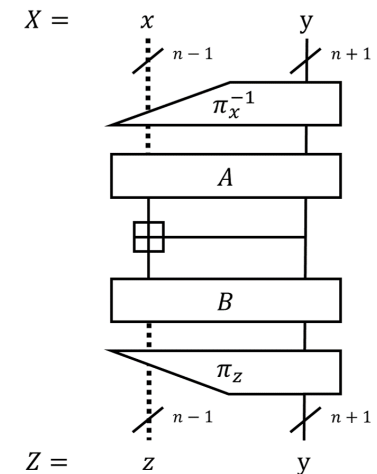
$$\begin{aligned}
 \pi_x \cdot x + \pi_z \cdot y + c_y = 0 &\implies \pi_x(x) = \pi_z(y) + c_y \\
 &= \pi_z(\mathbb{O}(x)) + c_y \\
 \implies \tilde{x} &= \pi_z(\mathbb{O}(\pi_x^{-1}(\tilde{x}))) + c_y \quad (\tilde{x} := \pi_x(x))
 \end{aligned}$$

가 성립하여 결과적으로 다음이 성립하게 된다.

$$X_{[0:n]} = \pi_z(\mathbb{O}(\pi_x^{-1}(X_{[0:n]}))) + c_y = Z_{[0:n]}$$

(새로운 오라클 $\mathbb{O}' = \pi_z \circ \mathbb{O} \circ \pi_x^{-1}$ 를 정의할 때, π_x^{-1}, π_z 를 $\mathbb{F}_2^N \rightarrow \mathbb{F}_2^N$ 으로 확장한다.)

본 절차의 목적이 right branch를 고정하는 것이므로 $n + 1$ 개 선형관계를 하위 비트에 적용한다.

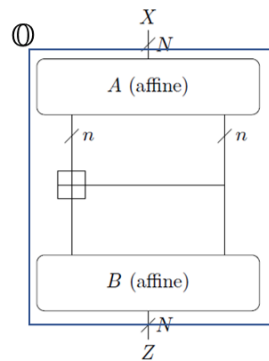


Locating Linear Bits

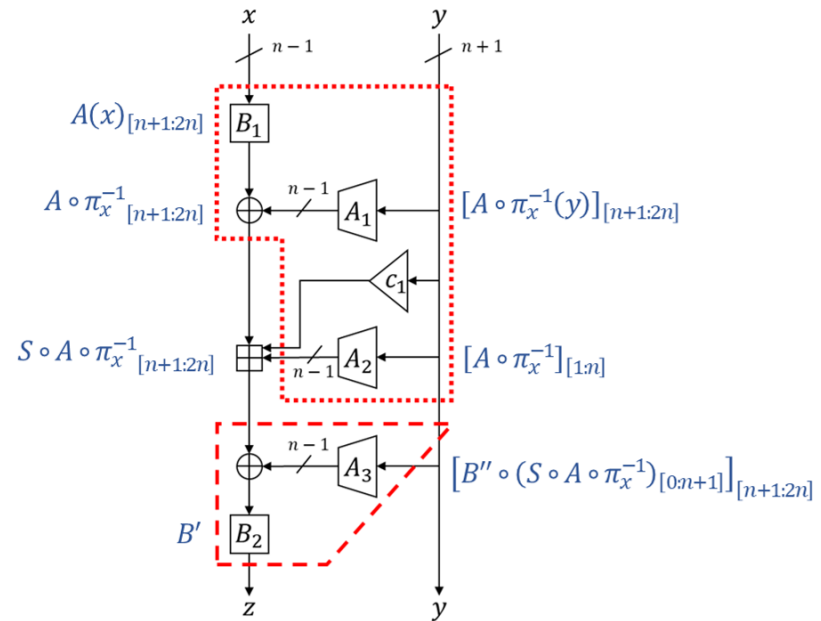
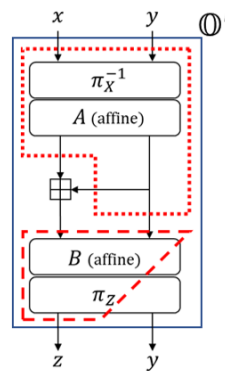
결과적으로 다음과 같은 새로운 오라클 $\mathbb{O}'$ 을 얻게 된다.

$$Z = \pi_z \circ \mathbb{O} \circ \pi_x^{-1}(X) = \pi_z \circ B \circ S \circ A \circ \pi_x^{-1}(X) = B' \circ S \circ A'(X) = \mathbb{O}'(X)$$

정렬이 완료된 오라클의 구조는 다음과 같이 표현할 수 있다.



1



Locating Linear Bits

이때, π_z 는 B 에 의해 퍼진 $S(X)$ 의 고차 좌표를 정리해야 한다.

$X' \in \mathbb{F}_2^{2n}$ 에 대하여 $n+1$ 개 하위 비트는

$$\pi_x(X') = \pi_z(Z') = \pi_z(B \circ S \circ A(X'))$$

를 만족하므로, $X = A(X')$ 라고 할 때 $X' = A^{-1}(X)$ 이고,

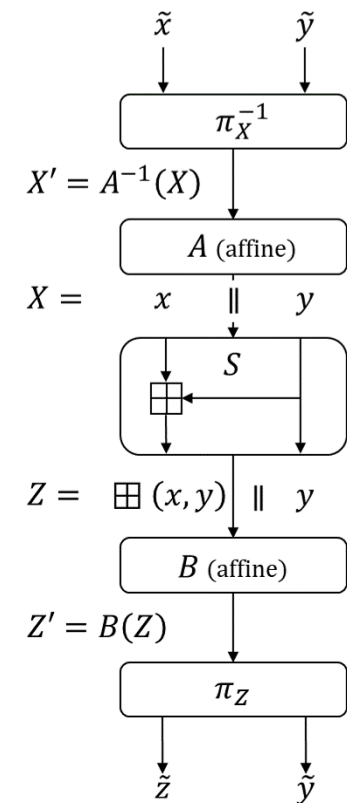
$$\pi_x \circ A^{-1}(X) = \pi_z \circ B \circ S(X)$$

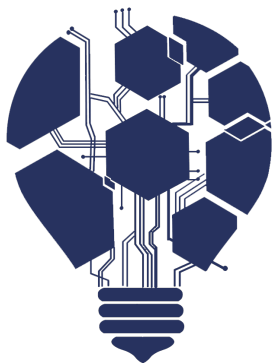
이다.

$$\begin{pmatrix} \tilde{x} \\ \tilde{y} \end{pmatrix} = \begin{pmatrix} (\pi_x)_1 & (\pi_x)_2 \\ (\pi_x)_3 & (\pi_x)_4 \end{pmatrix} \begin{pmatrix} A_1^{-1} & A_2^{-1} \\ A_3^{-1} & A_4^{-1} \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} \implies \tilde{y} = ((\pi_x)_3 \ (\pi_x)_4) \begin{pmatrix} A_1^{-1}x + A_2^{-1}y \\ A_3^{-1}x + A_4^{-1}y \end{pmatrix}$$

$$\begin{pmatrix} \tilde{z} \\ \tilde{y} \end{pmatrix} = \begin{pmatrix} (\pi_z)_1 & (\pi_z)_2 \\ (\pi_z)_3 & (\pi_z)_4 \end{pmatrix} \begin{pmatrix} B_1 & B_2 \\ B_3 & B_4 \end{pmatrix} \begin{pmatrix} \boxplus \\ y \end{pmatrix} \implies \tilde{y} = ((\pi_z)_3 \ (\pi_z)_4) \begin{pmatrix} B_1 \boxplus + B_2 y \\ B_3 \boxplus + B_4 y \end{pmatrix}$$

$$\implies (\pi_x)_3 A_1^{-1}x + (\pi_x)_3 A_2^{-1}y + (\pi_x)_4 A_3^{-1}x + (\pi_x)_4 A_4^{-1}y = (\pi_z)_3 B_1 \boxplus + (\pi_z)_3 B_2 y + (\pi_z)_4 B_3 \boxplus + (\pi_z)_4 B_4 y$$





- ① 1: Locating Linear Bits
- ② 2: Triangularization of the outer affine maps
- ③ 3: Recovery of the outer affine maps (left branches)
- ④ 4: Recovery of the Feistel affine maps (right to left XOR and addition)
- ⑤ 5: Correction of the first carry bit

Triangularization of the outer affine maps

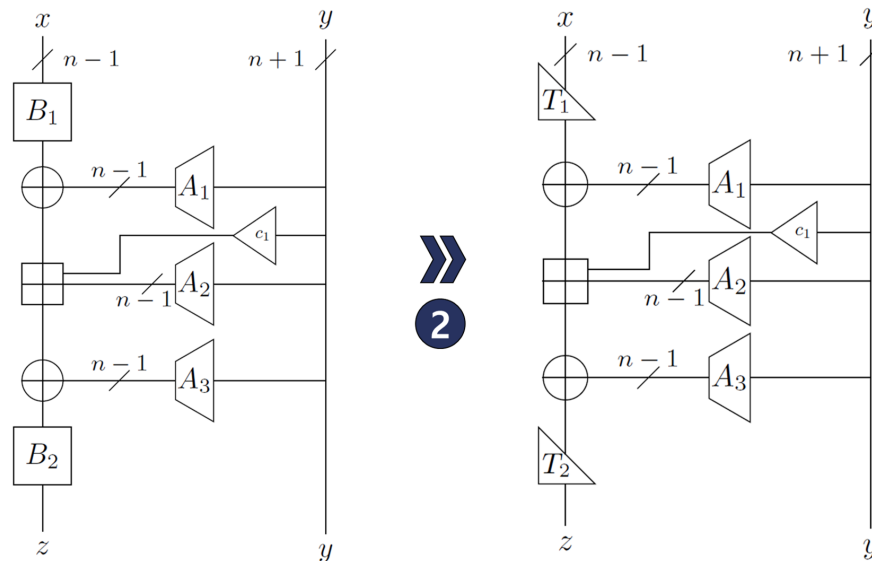
단계 2와 단계 3의 공격 목표는 1단계에서 정리한 라운드 함수 구조에서 affine map B_1, B_2 를 제거하여 left branch에 대하여 XOR과 모듈러 덧셈이 라운드 입출력의 좌표계에서 수행되도록 하는 것이다.

단, 한 번에 identity map으로 복원하지는 못하고,

알고리즘 2를 수행해서 구한 새로운 linear map π_x, π_z 을 추가 합성하면 B_1, B_2 가 하삼각 구조 T_1, T_2 가 된다.

T_1, T_2 는 6.3장에서 마저 복원한다.

하삼각행렬을 만들기 위한 알고리즘이 아니라 B_1, B_2 를 복구하기 위한 알고리즘을 수행하고 나면 하삼각행렬 형태가 되는 것으로 보임



Triangularization of the outer affine maps

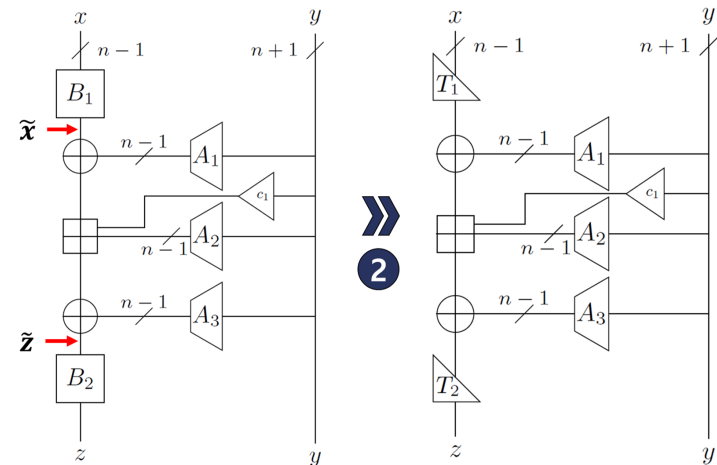
단계 2는 affine map B_1, B_2 를 완전히 제거하기 앞서 하삼각행렬로 만들 수 있는 linear map π_x, π_z 를 구한다.

이번 단계의 공격 대상은 left-branch에 의존하므로 y 를 0으로 고정한 상태로 x 를 바꿔가며 입력을 샘플링한다.

이때, π_x, π_z 를 구하기 위해 입력 차분의 하위 비트부터 순차적으로 상수로(0으로) 고정해가며 활성화되는 입력 범위를 축소시켜 공격을 수행한다. (Algorithm 2의 Step 4)

오라클의 입력은 랜덤하게 생성한 차분 $\Delta x = (*^{(n-1)-k} || 0^{(n+1)+k})$ 를 만족하는 $n + \epsilon$ 개 (x, x') 쌍이고, 샘플링 집합 Z 는 $\{(x, x')\}$ 으로 구한 $n + \epsilon$ 개 $\mathbb{O}_k(\Delta x) = \mathbb{O}(x) \oplus \mathbb{O}(x')$ 으로 구성한다.

Algorithm 2는 $\text{rank } Z = (n - 1) - k$ 이면 $(\Delta \tilde{x})_k = (\Delta \tilde{z})_k = 1$ 이라는 점을 활용한다. (Proposition 3)



Algorithm 2 Triangularization of the outer maps on the left branch

Input: oracle $\mathbb{O}$ implementing bijective modular addition with partially recovered affine encodings (Subsection 6.1)

Output: affine maps π_x, π_z such that $\mathbb{O}' = \pi_z \circ \mathbb{O} \circ \pi_x^{-1}$ matches structure from Figure 8 with lower triangular T_1, T_2

Complexity: $\mathcal{O}(n^2)$ queries, $\mathcal{O}(n^4)$ time

- 1: $\mathbb{O}_0 \leftarrow \mathbb{O}$
- 2: **for** $k \in \{0, \dots, n - 2\}$ **do**
- 3: **repeat**
- 4: $\Delta x \xleftarrow{\$} (*^{n-1-k} || 0^{n+1+k})$
- 5: $Z \leftarrow \left\{ \Delta \mathbb{O}_k(\Delta x) \mid n + \epsilon \text{ times} \right\}$
- 6: **until** $\text{rank } Z = n - 1 - k$

Triangularization of the outer affine maps

Proposition 3. Let $\Delta x = (\cdots \parallel 0^k)$, $\Delta y = 0^n$. Let Z denote the set of all possible output differences Δz of $z = x \boxplus y$. Then, $\text{rank } Z \leq n - k$. Furthermore, if $\text{rank } Z = n - k$, then $(\Delta x)_k = (\Delta z)_k = 1$, i.e., Δx and Δz both have shape $(\cdots \parallel 1 \parallel 0^k)$.

$\Delta x = x \oplus x'$, $\Delta y = y \oplus y' = 0$ 라고 하자. 그렇다면 $\Delta z = (x \boxplus y) \oplus (x' \boxplus y')$ 이므로,

$$\begin{aligned} \Delta z &= (x_{n-1} \oplus y_{n-1} \oplus c_{n-1}, \dots, x_0 \oplus y_0 \oplus c_0) \oplus (x'_{n-1} \oplus y'_{n-1} \oplus c'_{n-1}, \dots, x'_0 \oplus y'_0 \oplus c'_0) \\ &= (x_{n-1} \oplus x'_{n-1} \oplus c_{n-1} \oplus c'_{n-1}, \dots, x_0 \oplus x'_0 \oplus c_0 \oplus c'_0) \\ &= (\Delta x_{n-1} \oplus \Delta c_{n-1}, \dots, \Delta x_0 \oplus \Delta c_0) \quad (\Delta c_j := c_j \oplus c'_j) \end{aligned}$$

이때 $x_0 = x'_0, \dots, x_{k-1} = x'_{k-1}$ 이고 $c_0 = c'_0, \dots, c_k = c'_k$ 이기 때문에,

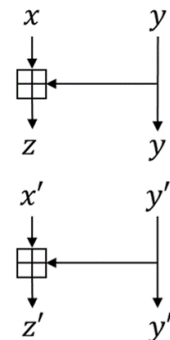
$\Delta z_i = (\Delta x_{n-1} \oplus \Delta c_{n-1}, \dots, \Delta x_{k+1} \oplus \Delta c_{k+1}, \Delta x_k, 0, \dots, 0)$ 이므로 $\text{rank } Z \leq n - k$ 는 자명하다.

($\Delta y_{k-1} = 0$ 일 때 $(x_{k-1} \boxplus y_{k-1}) = (x'_{k-1} \boxplus y'_{k-1})$ 이므로 $\Delta c_k = 0$)

한편, $\Delta x_k \neq 1$ 라고 가정하면 (즉, $\Delta x_k = 0$ 라면) $\Delta z_k = 0$ 이므로 $\text{rank } Z \leq n - (k + 1)$ 이 되어 $\text{rank } Z$ 는 $n - k$ 가 될 수 없다.

$\therefore \text{rank } Z = n - k$ 이면, $\Delta x_k = 1$ 이고 또한 $\Delta z_k = 1$ 이다. □

역은 성립하지 않지만, 본 논문에서는 평균적으로 $O(n)$ 번 샘플링할 때 $\text{rank } Z = n - k$ 가 된다고 가정한다.

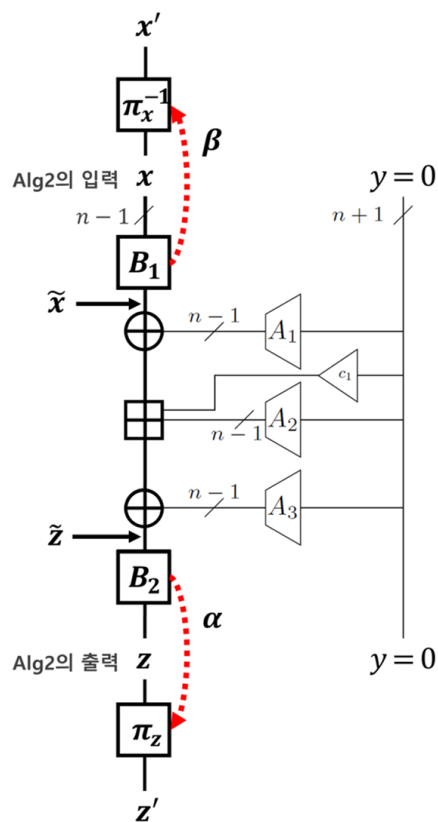


Triangularization of the outer affine maps: $B_2 \rightarrow T_2$

```

7:   solve  $Z \times \alpha = (1, \dots, 1)^T$  for  $\alpha = (*^{n-1-k} \parallel 0^{n+1+k})$ 
8:    $\pi_z \leftarrow$  a linear map such that  $(\pi_z(z))_{n+1+k} = \langle \alpha, z \rangle$ ,  $(\pi_z(z))_i = z_i$  for  $i > n+1+k$ 
9:    $\beta \leftarrow (\pi_z(\Delta \mathbb{O}_k(e_i))_{n+1+k} \mid n+1+k \leq i < 2n) \parallel 0^{n+1+k}$ 
10:   $\pi_x \leftarrow$  a linear map such that  $(\pi_x(x))_{n+1+k} = \langle \beta, x \rangle$ ,  $(\pi_x(x))_i = x_i$  for  $i > n+1+k$ 
11:   $\mathbb{O}_{k+1} \leftarrow \pi_z \circ \mathbb{O} \circ \pi_x^{-1}$ 
12: end for
13: return  $\mathbb{O}_{n-1}$ 

```



$\text{rank}\{\Delta \tilde{z}\} = n - 1 - k$: full-rank

$$\implies \Delta \tilde{x}_{n+1} \oplus \Delta \tilde{z}_{n+1} = 1$$

for $k = 0$

$$x' = (x_{2n-1}, \dots, x_{n+2}, \langle \beta^{(0)}, x \rangle, x_n, \dots, x_0)$$

$$z' = (z_{2n-1}, \dots, z_{n+2}, \langle \alpha^{(0)}, z \rangle, z_n, \dots, z_0)$$

$\mathbb{O} \parallel$

$$\langle \alpha^{(0)}, z \rangle = \langle \alpha^{(0)}, \mathbb{O}_0(x) \rangle = \sum_{j=0}^{n-2} \alpha_{n+1+j}^{(0)} \mathbb{O}_0(x)_{n+1+j}$$

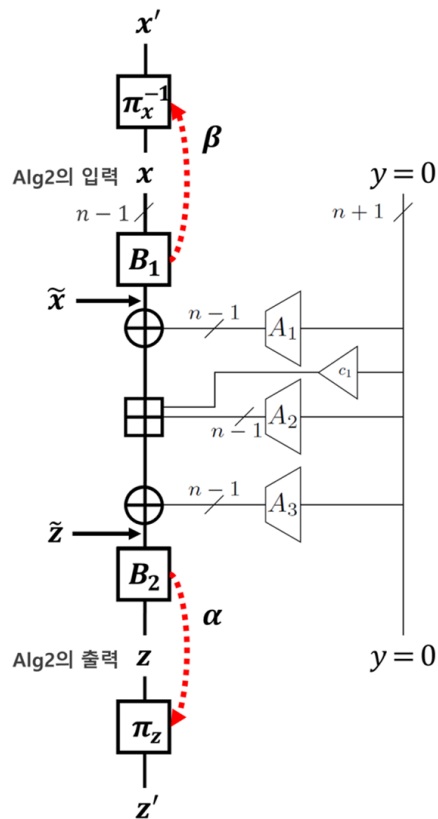
$$\langle \beta^{(0)}, x \rangle = \begin{cases} 1 & \text{if } \mathbb{O}_0(x)_{n+1} = 1 \\ 0 & \text{if } \mathbb{O}_0(x)_{n+1} = 0 \end{cases}$$

Triangularization of the outer affine maps: $B_2 \rightarrow T_2$

```

7:   solve  $Z \times \alpha = (1, \dots, 1)^T$  for  $\alpha = (*^{n-1-k} \parallel 0^{n+1+k})$ 
8:    $\pi_z \leftarrow$  a linear map such that  $(\pi_z(z))_{n+1+k} = \langle \alpha, z \rangle$ ,  $(\pi_z(z))_i = z_i$  for  $i > n+1+k$ 
9:    $\beta \leftarrow (\pi_z(\Delta \mathbb{O}_k(e_i))_{n+1+k} \mid n+1+k \leq i < 2n) \parallel 0^{n+1+k}$ 
10:   $\pi_x \leftarrow$  a linear map such that  $(\pi_x(x))_{n+1+k} = \langle \beta, x \rangle$ ,  $(\pi_x(x))_i = x_i$  for  $i > n+1+k$ 
11:   $\mathbb{O}_{k+1} \leftarrow \pi_z \circ \mathbb{O} \circ \pi_x^{-1}$ 
12: end for
13: return  $\mathbb{O}_{n-1}$ 

```



$\alpha^{(0)}$ 는 Δx 로 샘플링한 모든 $\Delta \tilde{z}$ 에 대하여 $\langle \alpha^{(0)}, \Delta \tilde{z} \rangle = 1$ 를 만족함

$$\implies \langle \alpha^{(0)}, B_2 \Delta \tilde{x} \rangle = 1 = \langle B_2^T \alpha^{(0)}, \Delta \tilde{x} \rangle \quad \because \alpha^{(0)T} (B_2 \Delta \tilde{x}) = (\alpha^{(0)T} (B_2 \Delta \tilde{x}))^T = \Delta \tilde{x}^T (B_2^T \alpha^{(0)})$$

이때 모든 $\Delta \tilde{z}$ 는 $\Delta \tilde{z}_{n+1} = 1$ ($k=0$) 이므로

$$B_2^T \alpha^{(0)} = e_{n+1} \implies \alpha^{(0)} = (B_2^T)^{-1} e_{n+1} \implies (B_2^T)^{-1} \text{의 } n+1 \text{ 번째 열 벡터} \\ \implies B_2^{-1} \text{의 } n+1 \text{ 번째 행 벡터}$$

한편,

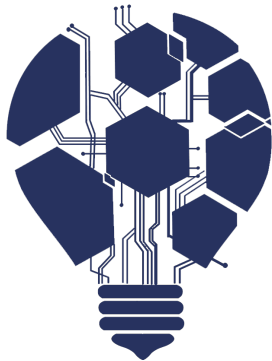
$$\begin{aligned} \langle B_2^T \alpha^{(0)}, \Delta \tilde{x} \rangle &= \langle e_{n+1}, \Delta \tilde{x} \rangle = \Delta \tilde{x}_{n+1} = \Delta \tilde{z}_{n+1} \quad (\text{for } k=0) \text{ 이고,} \\ &= \Delta B_1(x)_{n+1} \\ &= B_1(\Delta x)_{n+1} = \langle B_1 \Delta x, e_{n+1} \rangle = \langle \Delta x, B_1^T e_{n+1} \rangle \end{aligned}$$

이때 $B_1^T e_{n+1}$ 는 B_1 의 $n+1$ 번째 행 벡터이므로 $\implies \beta^{(0)}$

Triangularization of the outer affine maps: $B_2 \rightarrow T_2$

k 를 증가시키며 오라클의 left branch outer affine map들을 복구해 나간다.

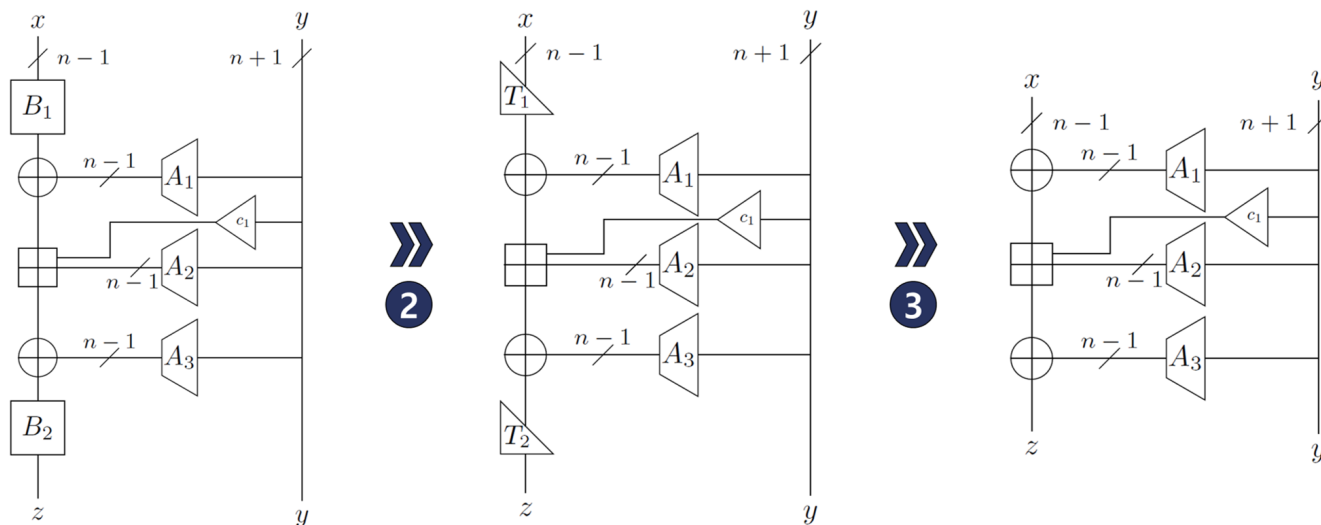
이때, 입력 비트를 한 비트씩 줄여가며 α 와 β 를 구하기 때문에,
단계 2에서 복구한 T_1, T_2 의 최종 형태는 삼각행렬이다.



- ① 1: Locating Linear Bits
- ② 2: Triangularization of the outer affine maps
- ③ 3: Recovery of the outer affine maps (left branches)
- ④ 4: Recovery of the Feistel affine maps (right to left XOR and addition)
- ⑤ 5: Correction of the first carry bit

Recovery of the outer affine maps

단계 3에서는 단계 2에 이어 left branch의 Affine map을 제거하는 단계이다.
 이때, 단계 3의 목표는 모듈러 덧셈의 차분 특성을 사용하여
 단계 2에서 일부 복원한 하삼각 구조 T_1, T_2 를 완전히 복구하는 것이다.



단계 2에서 구한 오라클은 다음과 같은 특성을 가진다.

$$\begin{aligned}
 \Delta z &= \mathbb{O}(x, y) \oplus \mathbb{O}(x', y) \\
 &= T_2([\{T_1(x) \oplus A_1(y)\} \boxplus A_2(y)] \oplus A_3(y)) \oplus T_2([\{T_1(x') \oplus A_1(y)\} \boxplus A_2(y)] \oplus A_3(y)) \\
 &= T_2([\{T_1(x) \oplus A_1(y)\} \boxplus A_2(y)] \oplus A_3(y) \oplus [\{T_1(x') \oplus A_1(y)\} \boxplus A_2(y)] \oplus A_3(y)) \\
 &= T_2([\underbrace{\{T_1(x) \oplus A_1(y)\} \boxplus A_2(y)}_{\tilde{x}}] \oplus [\underbrace{\{T_1(x') \oplus A_1(y)\} \boxplus A_2(y)}_{\tilde{x}'}])
 \end{aligned}$$

Recovery of the outer affine maps

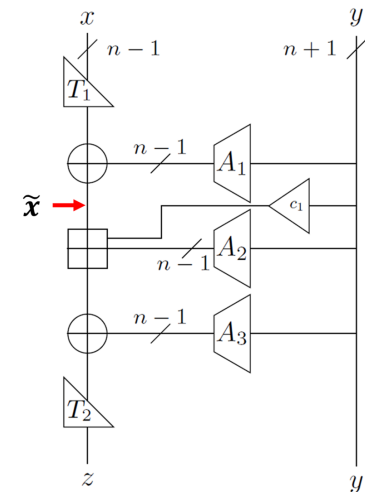
공격자는 오라클의 입력 x 의 $n-2$ 개 lsb를 제어할 때 출력 차분 Δz 를 sampling하고,

$$\Delta z = T_2([\underbrace{\{T_1(x) \oplus A_1(y)\}}_{\tilde{x}} \boxplus A_2(y)] \oplus [\underbrace{\{T_1(x') \oplus A_1(y)\}}_{\tilde{x}'} \boxplus A_2(y)])$$

T_1, T_2 는 가역 affine map이므로, Proposition 4를 이용하여

$$\max \Pr[(\Delta x, \Delta y) \rightarrow \Delta z] = \begin{cases} 1/2, & \text{if } \tilde{x} \oplus \tilde{x}' = \Delta x_1 \\ 1/4, & \text{if } \tilde{x} \oplus \tilde{x}' = \Delta x_2 \end{cases}$$

라고 판단한다.



Proposition 4. Let $z = x \boxplus y$ be an n -bit modular addition, $n \geq 3$. Set

$$\Delta y = 0, \quad \Delta x_1 = e_0 = (0, 0, 0, \dots, 0, 1), \quad \Delta x_2 = e_0 \oplus e_{n-2} = (0, 1, 0, \dots, 0, 1).$$

Then, the most probable transitions with input differences $(\Delta x_1, \Delta y)$ and $(\Delta x_2, \Delta y)$ respectively are described by

$$\Pr[(\Delta x_1, \Delta y) \xrightarrow{\boxplus} \Delta z] = \begin{cases} 1/2 & \text{if } \Delta z = (0, \dots, 0, 0, 1) = \Delta x_1, \\ 1/4 & \text{if } \Delta z = (0, \dots, 0, 1, 1), \\ \leq 1/4 & \text{otherwise.} \end{cases}$$

$$\Pr[(\Delta x_2, \Delta y) \xrightarrow{\boxplus} \Delta z] = \begin{cases} 1/4 & \text{if } \Delta z = (0, 1, \dots, 0, 1) = \Delta x_2, \\ 1/4 & \text{if } \Delta z = (1, 1, \dots, 0, 1) = \Delta x_2, \\ \leq 1/4 & \text{otherwise.} \end{cases}$$

Recovery of the outer affine maps

단계 3은 Proposition 4를 증명하기 위해 5.1장의 Lemma1과 Theorem1을 이용하며, 이는 Helger Lipmaa와 Shiho Moriai의 논문을 인용한 것이다.[2]

Theorem 1. of L.M. Let $\delta = (\alpha, \beta \mapsto \gamma)$ be an arbitrary differential. Algorithm 2 returns $DP^+(\delta)$ in time $\Theta(\log n)$. More precisely, it works in time $\Theta(1) + t$, where t is the time it takes to compute w_h .

Algorithm 2. of L.M. INPUT: $\delta = (\alpha, \beta \mapsto \gamma)$

- ① If $\text{eq}(\alpha \ll 1, \beta \ll 1, \gamma \ll 1) \wedge (\text{xor}(\alpha, \beta, \gamma) \oplus (\beta \ll 1)) \neq 0$ then return 0;
- ② Return $2^{-w_h(\neg \text{eq}(\alpha, \beta, \gamma) \wedge \text{mask}(n-1))}$.

Efficient Algorithms for Computing Differential Properties of Addition

Helger Lipmaa¹ and Shiho Moriai²

¹ Helsinki University of Technology, Department of Computer Science and Engineering
P.O.Box 5400, FI-02015 HUT, Espoo, Finland
helger@cs.hut.fi

² NTT Laboratories

1-1 Hikari-no-oka, Yokosuka, 239-0847 Japan
shiho@isl.ntt.co.jp

Abstract. In this paper we systematically study the differential properties of addition modulo 2^n . We derive $\Theta(\log n)$ -time algorithms for most of the properties, including differential probability of addition. We also present log-time algorithms for finding good differentials. Despite the apparent simplicity of modular addition, the best known algorithms require naive exhaustive computation. Our results represent a significant improvement over them. In the most extreme case, we present a complexity reduction from $\Omega(2^{4n})$ to $\Theta(\log n)$.

Keywords: modular addition, differential cryptanalysis, differential probability, impossible differentials, maximum differential probability.

1 Introduction

One of the most successful and influential attacks against block ciphers is Differential Cryptanalysis (DC), introduced by Biham and Shamir in 1991 [BS91a]. For many of the block ciphers proposed since then, provable security against DC (defined by Lai, Massey and Murphy [LMM91] and first implemented by Nyberg and Knudsen [NK95]) has been one of the primary criteria used to confirm their potential quality.

Unfortunately, few approaches to proving security have been really successful. The original approach of [NK95] has been used in designing MISTY and its variant KASUMI (the new 3GPP block cipher standard). Another influential approach has been the “wide trail” strategy proposed by Daemen [Dae95], applied for example in the proposed AES, Rijndael. The main reason for the small number of successful strategies is the complex structure of modern ciphers, which makes exact evaluation of their differential properties infeasible. This has, unfortunately, led to a situation where the security against DC is often evaluated by heuristic methods.

We approach the above problem by using the bottom-up methodology. That is, we evaluate many sophisticated differential properties of one of the most-used “non-trivial” block cipher cornerstones, *addition modulo 2^n* for $n \geq 1$. We hope that this will help to evaluate the differential properties of larger composite cipher parts like the Pseudo-Hadamard Transform, with the entire cipher being the final goal. The algorithms proposed here will enable the advanced cryptanalysis of block ciphers. We hope that our

Recovery of the outer affine maps - Proof of Theorem 1 of [2]

인용된 Lipmaa 논문의 Theorem 1을 증명하기 위해, 먼저 해당 논문의 Lemma 2와 Lemma 3을 증명한다.

Lemma 2. of L.M. Let $L(x)$ be a mapping, such that

$$L(x) = \begin{cases} 0, & x = 0, \\ 1/2, & x = 1, 2, \\ 1, & x = 3. \end{cases}$$

Let $\alpha, \beta \in \sum^n$. Then

$$\Pr_{x,y} [\text{carry}(x, y)_{i+1} \oplus \text{carry}(x \oplus \alpha, y \oplus \beta)_{i+1} = 1 \mid \alpha_i + \beta_i + \Delta c_i = j] = L(j).$$

Proof) x, y 가 주어졌을 때, $c := \text{carry}(x, y)$, $c' := \text{carry}(x \oplus \alpha, y \oplus \beta)$, $\Delta c = c \oplus c'$ 라고 하자. 그렇다면

$$\begin{aligned} \Delta c_{i+1} &= c_{i+1} \oplus c'_{i+1} \\ &= [(x_i \cdot y_i) \oplus (x_i \cdot c_i) \oplus (y_i \cdot c_i)] \oplus [((x \oplus \alpha)_i \cdot (y \oplus \beta)_i) \oplus ((x \oplus \alpha)_i \cdot c'_i) \oplus ((y \oplus \beta)_i \cdot c'_i)] \end{aligned}$$

이다. 다음으로 $f(\alpha_i, \beta_i, \Delta c_i) := \Pr_{x,y} [\Delta c_{i+1} := 1 \mid \alpha_i + \beta_i + \Delta c_i = j]$ 라고 하자.

이때, Δc_{i+1} 와 f 는 세 개의 쌍 (x, α) , (y, β) , $(c, \Delta c)$ 에 대하여 symmetric하다.

예를 들어, $(\alpha_i, \beta_i, \Delta c_i)$ 에 대하여 $f(0, 0, 1) = f(0, 1, 0) = f(1, 0, 0)$ 이며,

따라서 $\alpha_i + \beta_i + \Delta c_i = 1$ 에 대한 case는 이 중 하나의 case만 확인하면 된다.

이러한 성질을 활용하여 f 를 다음과 같이 재정의하고 각 case를 확인한다.

$$f(j) = \Pr_{x,y} [\Delta c_{i+1} = 1 \mid \alpha_i + \beta_i + \Delta c_i = j].$$

Recovery of the outer affine maps - Proof of Theorem 1 of [2]

① $j = 0$ 일 때,

$$\begin{aligned}
 f(0) &= \Pr_{x,y}[\Delta c_{i+1} = 1 \mid \alpha_i + \beta_i + \Delta c_i = 0] \\
 &= \Pr_{x,y}[\Delta c_{i+1} = 1 \mid (\alpha_i, \beta_i, \Delta c_i) = (0, 0, 0)] \\
 &= \Pr_{x,y} \left[[(x_i \cdot y_i) \oplus (x_i \cdot c_i) \oplus (y_i \cdot c_i)] \oplus [((x \oplus 0)_i \cdot (y \oplus 0)_i) \oplus ((x \oplus 0)_i \cdot c_i) \oplus ((y \oplus 0)_i \cdot c_i)] = 1 \right] \\
 &= 0.
 \end{aligned}$$

② $j = 1$ 일 때,

$$\begin{aligned}
 f(1) &= \Pr_{x,y}[\Delta c_{i+1} = 1 \mid \alpha_i + \beta_i + \Delta c_i = 1] \\
 &= \Pr_{x,y}[\Delta c_{i+1} = 1 \mid (\alpha_i, \beta_i, \Delta c_i) = (0, 0, 1)] \\
 &= \Pr_{x,y} \left[[(x_i \cdot y_i) \oplus (x_i \cdot c_i) \oplus (y_i \cdot c_i)] \oplus [((x \oplus 0)_i \cdot (y \oplus 0)_i) \oplus ((x \oplus 0)_i \cdot \neg c_i) \oplus ((y \oplus 0)_i \cdot \neg c_i)] = 1 \right] \\
 &= \Pr_{x,y}[(x_i \cdot c_i) \oplus (y_i \cdot c_i) \oplus (x_i \cdot \neg c_i) \oplus (y_i \cdot \neg c_i) = 1] \\
 &= \Pr_{x,y}[x_i \oplus y_i = 1] \quad (c_i = 0 \text{인 경우와 } c_i = 1 \text{인 경우 모두 성립)} \\
 &= \frac{1}{2}.
 \end{aligned}$$

Recovery of the outer affine maps - Proof of Theorem 1 of [2]

③ $j = 2$ 일 때,

$$\begin{aligned}
 f(2) &= \Pr_{x,y}[\Delta c_{i+1} = 1 \mid \alpha_i + \beta_i + \Delta c_i = 2] \\
 &= \Pr_{x,y}[\Delta c_{i+1} = 1 \mid (\alpha_i, \beta_i, \Delta c_i) = (1, 1, 0)] \\
 &= \Pr_{x,y}[(x_i \cdot y_i) \oplus (x_i \cdot c_i) \oplus (y_i \cdot c_i) \oplus [(x \oplus 1)_i \cdot (y \oplus 1)_i] \oplus ((x \oplus 1)_i \cdot c_i) \oplus ((y \oplus 1)_i \cdot c_i) = 1] \\
 &= \Pr_{x,y}[(x_i \cdot y_i) \oplus (x_i \cdot c_i) \oplus (y_i \cdot c_i) \oplus (\neg x_i \cdot \neg y_i) \oplus (\neg x_i \cdot c_i) \oplus (\neg y_i \cdot c_i) = 1] \\
 &= \Pr_{x,y}[(x_i \cdot y_i) \oplus (\neg x_i \cdot \neg y_i) \oplus (x_i \cdot c_i) \oplus (y_i \cdot c_i) \oplus (\neg x_i \cdot c_i) \oplus (\neg y_i \cdot c_i) = 1] \\
 &= \Pr_{x,y}[\neg(x_i \oplus y_i) \oplus 0 = 1] \quad (c_i = 0 \text{인 경우와 } c_i = 1 \text{인 경우 모두 } (x_i \cdot c_i) \oplus (y_i \cdot c_i) \oplus (\neg x_i \cdot c_i) \oplus (\neg y_i \cdot c_i) = 0) \\
 &= \Pr_{x,y}[x_i \oplus y_i = 0] \\
 &= \frac{1}{2}.
 \end{aligned}$$

Recovery of the outer affine maps - Proof of Theorem 1 of [2]

④ $j = 3$ 일 때,

$$\begin{aligned}
 f(3) &= \Pr_{x,y}[\Delta c_{i+1} = 1 \mid \alpha_i + \beta_i + \Delta c_i = 3] \\
 &= \Pr_{x,y}[\Delta c_{i+1} = 1 \mid (\alpha_i, \beta_i, \Delta c_i) = (1, 1, 1)] \\
 &= \Pr_{x,y}[(x_i \cdot y_i) \oplus (x_i \cdot c_i) \oplus (y_i \cdot c_i) \oplus ((x \oplus 1)_i \cdot (y \oplus 1)_i) \oplus ((x \oplus 1)_i \cdot \neg c_i) \oplus ((y \oplus 1)_i \cdot \neg c_i)] = 1] \\
 &= \Pr_{x,y}[(x_i \cdot y_i) \oplus (x_i \cdot c_i) \oplus (y_i \cdot c_i) \oplus (\neg x_i \cdot \neg y_i) \oplus (\neg x_i \cdot \neg c_i) \oplus (\neg y_i \cdot \neg c_i)] = 1] \\
 &= \Pr_{x,y}[\neg(x_i \oplus y_i) \oplus \neg(x_i \oplus c_i) \oplus \neg(y_i \oplus c_i) = 1] \\
 &= 1.
 \end{aligned}$$

① ~ ④가 모든 i 에 대하여 성립하므로, 결과적으로

$$f(j) = \Pr_{x,y} [\text{carry}(x, y)_{i+1} \oplus \text{carry}(x \oplus \alpha, y \oplus \beta)_{i+1} = 1 \mid \alpha_i + \beta_i + \Delta c_i = j] = \begin{cases} 0, & j = 0, \\ \frac{1}{2}, & j = 1, 2, \\ 1, & j = 3 \end{cases}$$

이므로, $\Pr_{x,y} [\text{carry}(x, y)_{i+1} \oplus \text{carry}(x \oplus \alpha, y \oplus \beta)_{i+1} = 1 \mid \alpha_i + \beta_i + \Delta c_i = j] = L(j)$ 이다. □

Recovery of the outer affine maps - Proof of Theorem 1 of [2]

Lemma 3. 1) of L.M. Every possible differential is "good".

가능한 차분은 다음을 만족하는 차분 $\delta = (\alpha, \beta \mapsto \gamma)$ 이다.

$$\text{DP}^+(\delta) = \Pr_{x,y}[(x \boxplus y) \oplus ((x \oplus \alpha) \boxplus (y \oplus \beta)) = \gamma] > 0$$

이때,

$$\begin{aligned} (x \boxplus y) \oplus ((x \oplus \alpha) \boxplus (y \oplus \beta)) &= (x \oplus y \oplus \text{carry}(x, y)) \oplus ((x \oplus \alpha) \oplus (y \oplus \beta) \oplus \text{carry}(x \oplus \alpha, y \oplus \beta)) \\ &= \text{carry}(x, y) \oplus \alpha \oplus \beta \oplus \text{carry}(x \oplus \alpha, y \oplus \beta) \\ &= \gamma \end{aligned}$$

이므로

$$\Pr_{x,y}[(x \boxplus y) \oplus ((x \oplus \alpha) \oplus (y \oplus \beta)) = \gamma] = \Pr_{x,y}[\text{carry}(x, y) \oplus \text{carry}(x \oplus \alpha, y \oplus \beta) = \alpha \oplus \beta \oplus \gamma]$$

이다.

즉, 어떤 δ 가 가능하다는 것은 모든 $i \in [0, n-1]$ 에 대하여

$$\Delta c_i = \text{carry}(x, y)_i \oplus \text{carry}(x \oplus \alpha, y \oplus \beta)_i = \alpha_i \oplus \beta_i \oplus \gamma_i$$

를 만족하는 (x, y) 가 적어도 하나 존재함을 의미한다.

Recovery of the outer affine maps - Proof of Theorem 1 of [2]

Lemma 3. 1) of L.M. Every possible differential is "good".

또한, Lipmaa 등은 차분 $\delta = (\alpha, \beta \mapsto \gamma)$ 가 다음을 만족할 때 "good"이라고 정의한다.

$$\text{eq}(\alpha \ll 1, \beta \ll 1, \gamma \ll 1) \wedge [\text{xor}(\alpha, \beta, \gamma) \oplus (\alpha \ll 1)] = 0.$$

즉,

$$\begin{aligned} \delta \text{ is not "good"} &\iff \text{eq}(\alpha \ll 1, \beta \ll 1, \gamma \ll 1) \wedge [\text{xor}(\alpha, \beta, \gamma) \oplus (\alpha \ll 1)] > 0 \\ &\iff \alpha_{i-1} = \beta_{i-1} = \gamma_{i-1} \neq \alpha_i \oplus \beta_i \oplus \gamma_i \quad \text{for some } i \in [0, n-1] \end{aligned}$$

이다. "good"은 모든 i 에 대하여 성립함을 보여야 하는데, "not good"은 하나의 i 만 찾으면 됨

Recovery of the outer affine maps - Proof of Theorem 1 of [2]

Lemma 3. 1) of L.M. Every possible differential is "good".

Proof) $\Delta c_i = \text{carry}(x, y)_i \oplus \text{carry}(x \oplus \alpha, y \oplus \beta)_i$ 라고 하자.

그리고 가능한 차분 δ 가 "not good"이라고 가정하자.

δ 는 가능한 차분이므로 모든 i 에 대하여 다음을 만족하는 (x, y) 가 존재한다.

$$\Pr_{x,y}[\Delta c_i = \alpha_i \oplus \beta_i \oplus \gamma_i] > 0$$

그리고 δ 가 "not good"이므로 $\alpha_{k-1} = \beta_{k-1} = \gamma_{k-1} \neq \alpha_k \oplus \beta_k \oplus \gamma_k$ 인 k 가 존재한다.

이때, $\alpha_{k-1} \oplus \beta_{k-1} \oplus \gamma_{k-1} = \gamma_{k-1}$ 이고, 따라서 $\Delta c_{k-1} = \gamma_{k-1}$ 이다.

Lemma 2에 의해, 이 k 에 대하여

$$\gamma_{k-1} = 0 \text{ 일 때, } \Pr_{x,y}[\Delta c_k = 1 \mid \alpha_{k-1} + \beta_{k-1} + \Delta c_{k-1} = 0] = 0,$$

$$\begin{aligned} \gamma_{k-1} = 1 \text{ 일 때, } \Pr_{x,y}[\Delta c_k = 0 \mid \alpha_{k-1} + \beta_{k-1} + \Delta c_{k-1} = 3] &= 1 - \Pr_{x,y}[\Delta c_k = 1 \mid \alpha_{k-1} + \beta_{k-1} + \Delta c_{k-1} = 3] \\ &= 1 - 1 = 0 \end{aligned}$$

이므로, $\Pr_{x,y}[\Delta c_k = \alpha_k \oplus \beta_k \oplus \gamma_k \neq \gamma_{k-1} \mid \alpha_{k-1} + \beta_{k-1} + \Delta c_{k-1} = j] = 0$ 이다. j 와 관계없이 확률이 0

이는 δ 가 가능한 차분이라는 가정에 모순이므로 가능한 차분 δ 는 "good" 이다. □

Recovery of the outer affine maps - Proof of Theorem 1 of [2]

Lemma 3. 2) of L.M. Let $\delta = (\alpha, \beta \mapsto \gamma)$ be "good". If $i \in [0, n-1]$, then $\Pr_{x,y} [\text{carry}(x, y)_i \oplus \text{carry}(x \oplus \alpha, y \oplus \beta)_i = 1 \mid \alpha_{i-1} + \beta_{i-1} + \Delta c_{i-1} = j] = L(j)$ possible. In particular, $\Pr_{x,y} [\text{carry}(x, y)_0 \oplus \text{carry}(x \oplus \alpha, y \oplus \beta)_0 = 0] = 1$.

Proof) δ 가 good이고, $\alpha_{-1} = \beta_{-1} = \gamma_{-1} = 0$ 이라고 하자.

그리고 x, y 가 주어졌을 때, $c := \text{carry}(x, y)$, $c' := \text{carry}(x \oplus \alpha, y \oplus \beta)$, $\Delta c = c \oplus c'$, $\Delta c_{-1} = 0$ 이라고 하자.

δ 가 good이면 가능한 차분임을 증명하기 위해 다음에 대한 수학적 귀납법을 사용한다.

$$\Pr_{x,y} [\Delta c = \alpha \oplus \beta \oplus \Delta c \pmod{2^i}] > 0$$

① $i = 0$ 일 때,

$$\Pr_{x,y} [c_0 \oplus c'_0 = 1 \mid \alpha_{-1} + \beta_{-1} + \gamma_{-1} = 0] = L(0)$$

이고, 특히 carry의 정의에 의해 $c_0 = c'_0 = 0$ 이므로 $\Pr_{x,y} [c_0 \oplus c'_0 = 0] = 1 > 0$ 이다.

Recovery of the outer affine maps - Proof of Theorem 1 of [2]

② $i \geq 1$ 일 때,

먼저, Lemma 2의 L 은 $a_1, a_2, a_3 \in \mathbb{F}_2$ 에 대하여 $L(a_1 + a_2 + a_3) = L(a_1 + a_2 + (a_1 \oplus a_2 \oplus a_3))$ 이다.

㉠ $a_1 = a_2$ 이면, $a_1 \oplus a_2 \oplus a_3 = a_3$ 이므로 $L(a_1 + a_2 + a_3) = L(a_1 + a_2 + (a_1 \oplus a_2 \oplus a_3))$ 이다.

㉢ $a_1 \neq a_2$ 이면, $a_1 + a_2 = 1$ 이고, $a_1 \oplus a_2 \oplus a_3 = \neg a_3$ 이므로

$$L(a_1 + a_2 + a_3) = L(1 + a_3) = \frac{1}{2} = L(1 + \neg a_3) = L(a_1 + a_2 + (a_1 \oplus a_2 \oplus a_3)) \text{이다.}$$

이제 i 에 대하여 $\Pr_{x,y} [\Delta c = \alpha \oplus \beta \oplus \gamma \pmod{2^i}] > 0$ 가 성립한다고 가정하자.

특히 $\Delta c_{i-1} = (\alpha \oplus \beta \oplus \gamma)_{i-1}$ 인 (x, y) 가 존재한다. 그러면 Lemma 2에 의해

$$\begin{aligned} \Pr_{x,y} [\Delta c_i = 1 \mid \alpha_{i-1} + \beta_{i-1} + \gamma_{i-1} = j] &= \Pr_{x,y} [\Delta c_i = 1 \mid \alpha_{i-1} + \beta_{i-1} + (\alpha \oplus \beta \oplus \gamma)_{i-1} = j] \\ &= \Pr_{x,y} [\Delta c_i = 1 \mid \alpha_{i-1} + \beta_{i-1} + \Delta c_{i-1} = j] \\ &= L(j) \end{aligned}$$

이다.

논문의 증명은 여기서 마무리됨

Recovery of the outer affine maps - Proof of Theorem 1 of [2]

또한,

$$\Pr_{x,y} [\Delta c_i = \alpha \oplus \beta \oplus \gamma \mid \alpha_{i-1} + \beta_{i-1} + \Delta c_{i-1} = j] = \begin{cases} L(j), & \alpha \oplus \beta \oplus \gamma = 1, \\ 1 - L(j), & \alpha \oplus \beta \oplus \gamma = 0 \end{cases}$$

이다.

이때, δ 가 good이므로 j 와 관계없이

$$\Pr_{x,y} [\Delta c_i = \alpha \oplus \beta \oplus \gamma \mid \alpha_{i-1} + \beta_{i-1} + \Delta c_{i-1} = j] > 0$$

이고, 따라서

$$\Pr_{x,y} [\Delta c = \alpha \oplus \beta \oplus \gamma \pmod{2^{i+1}}] > 0$$

이다.

이를 통해 $i + 1$ 에 대하여 반복할 수 있고, $i = n - 1$ 까지 반복하면

$$\Pr_{x,y} [\Delta c = \alpha \oplus \beta \oplus \gamma \pmod{2^n}] > 0$$

이다. 결과적으로, good인 δ 는 가능한 차분이다. □

Recovery of the outer affine maps - Proof of Theorem 1 of [2]

Theorem 1. of L.M. Let $\delta = (\alpha, \beta \mapsto \gamma)$ be an arbitrary differential. Algorithm 2 returns $DP^+(\delta)$ in time $\Theta(\log n)$. More precisely, it works in time $\Theta(1) + t$, where t is the time it takes to compute w_h .

Algorithm 2. of L.M. INPUT: $\delta = (\alpha, \beta \mapsto \gamma)$

- ① If $\text{eq}(\alpha \ll 1, \beta \ll 1, \gamma \ll 1) \wedge (\text{xor}(\alpha, \beta, \gamma) \oplus (\beta \ll 1)) \neq 0$ then return 0;
- ② Return $2^{-w_h(\neg \text{eq}(\alpha, \beta, \gamma) \wedge \text{mask}(n-1))}$;

Proof) Lemma 3에 의해 가능한 차분 δ 는 "good"이다. 가능한 차분 δ 에 대한 확률은

$$DP^+(\delta) = \prod_{i=0}^{n-2} \Pr_{x,y} [\Delta c_{i+1} = \alpha_{i+1} \oplus \beta_{i+1} \oplus \gamma_{i+1} \mid \Delta c_i = \alpha_i \oplus \beta_i \oplus \gamma_i]$$

이고, Lemma 3.2)와 Lemma 2에 의해

$$\Pr_{x,y} [\Delta c_{i+1} = \alpha_{i+1} \oplus \beta_{i+1} \oplus \gamma_{i+1} \mid \Delta c_i = \alpha_i \oplus \beta_i \oplus \gamma_i] = \begin{cases} 1 & \text{if } \alpha_{i+1} = \beta_{i+1} = \gamma_{i+1}, \\ \frac{1}{2} & \text{otherwise} \end{cases}$$

이다. 따라서

$$DP^+(\delta) = \left(\frac{1}{2}\right)^{\sum_{i=0}^{n-2} \neg \text{eq}(\alpha_i, \beta_i, \gamma_i)} = 2^{-w_h(\neg \text{eq}(\alpha, \beta, \gamma) \wedge \text{mask}(n-1))}$$

이다. □

Recovery of the outer affine maps - Proof of Proposition 4

증명한 [2]의 Theorem 등을 변형하면, Biryukov 등의 논문 [1] 5.1장의 Lemma 1과 Theorem 1을 얻을 수 있다.

Lemma 1. The probability of a differential transition $(\alpha, \beta) \rightarrow \gamma$ through modular addition is nonzero if and only if

$$\alpha_i \oplus \beta_i \oplus \gamma_i = \begin{cases} 0 & \text{if } (i = 0), \\ \beta_{i-1} & \text{if } (i \geq 1) \wedge (\alpha_{i-1} = \beta_{i-1} = \gamma_{i-1}). \end{cases}$$

Theorem 1. If differential transition $(\alpha, \beta) \rightarrow \gamma$ through modular addition has nonzero probability, then the probability is equal to 2^{-n+l+1} , where

$$l = |i \in \{0, \dots, n-2\} : \alpha_i = \beta_i = \gamma_i|.$$

이제 Lemma 1과 Theorem 1을 이용하여 6.3장 단계 3의 Proposition 4를 증명한다.

Recovery of the outer affine maps - Proof of Proposition 4

Proposition 4. Let $z = x \boxplus y$ be an n -bit modular addition, $n \geq 3$. Set

$$\Delta y = 0, \quad \Delta x_1 = e_0 = (0, 0, 0, \dots, 0, 1), \quad \Delta x_2 = e_0 \oplus e_{n-2} = (0, 1, 0, \dots, 0, 1).$$

Then, the most probable transitions with input differences $(\Delta x_1, \Delta y)$ and $(\Delta x_2, \Delta y)$ respectively are described by

$$\Pr[(\Delta x_1, \Delta y) \xrightarrow{\boxplus} \Delta z] = \begin{cases} 1/2 & \text{if } \Delta z = (0, \dots, 0, 0, 1) = \Delta x_1, \\ 1/4 & \text{if } \Delta z = (0, \dots, 0, 1, 1), \\ \leq 1/4 & \text{otherwise.} \end{cases}$$

$$\Pr[(\Delta x_2, \Delta y) \xrightarrow{\boxplus} \Delta z] = \begin{cases} 1/4 & \text{if } \Delta z = (0, 1, \dots, 0, 1) = \Delta x_2, \\ 1/4 & \text{if } \Delta z = (1, 1, \dots, 0, 1) = \Delta x_2, \\ \leq 1/4 & \text{otherwise.} \end{cases}$$

Recovery of the outer affine maps - Proof of Proposition 4

Proof) Lemma 1의 $\alpha := \Delta x$, $\beta := \Delta y = 0$, $\gamma := \Delta z$ 로 두자. Theorem 1에 의해 가능한 δ 에 대하여

$$\text{DP}^+(\alpha, \beta \rightarrow \gamma) = 2^{-n+l+1} = 2^{-w_h(\neg \text{eq}(\Delta x, \Delta y, \Delta z) \wedge \text{mask}(n-1))}$$

이고, 이때 모든 $i \in [0, n-1]$ 에 대하여 $\Delta x_{i-1} = \Delta y_{i-1} = \Delta z_{i-1}$ 이면 $\Delta x_i \oplus \Delta y_i \oplus \Delta z_i = \Delta x_{i-1}$ 이다.

가정에서 $\Delta y = 0$ 이므로 $\Delta x_{i-1} = \Delta y_{i-1} = \Delta z_{i-1}$ 은 $\Delta x_{i-1} = \Delta z_{i-1} = 0$ 과 동치이고, 따라서 $\neg \text{eq}(\Delta x, 0, \Delta z) = \Delta x \vee \Delta z$ 이다.

즉, 가능한 차분에 대하여

$$\text{DP}^+(\Delta x, 0 \rightarrow \Delta z) = 2^{-w_h((\Delta x \vee \Delta z) \wedge \text{mask}(n-1))}$$

이고 이때 모든 $i \in [0, n-1]$ 에 대하여 $\Delta x_{i-1} = \Delta z_{i-1} = 0$ 이면 $\Delta x_i = \Delta z_i$ 이다.

또한 $w_h((\Delta x \vee \Delta z) \wedge \text{mask}(n-1)) \geq (w_h(\Delta x) \wedge \text{mask}(n-1))$ 이므로

$$\text{DP}^+(\Delta x, 0 \rightarrow \Delta z) \leq 2^{-w_h(\Delta x \wedge \text{mask}(n-1))}$$

이고, 등호는 Δz 가 마스킹 범위인 $[0, n-2]$ 안에서 Δx 와 차이가 생기는 새로운 1을 만들지 않을 때에만 성립한다.

Recovery of the outer affine maps - Proof of Proposition 4

① $\Delta x = \Delta x_1 = e_0$ 의 경우,

$w_h(\Delta x \wedge \text{mask}(n-1)) = 1$ 이므로 모든 Δz 에서 $\Pr[\Delta x, 0 \rightarrow \Delta z] \leq \frac{1}{2}$ 이다.

$\Delta z = e_0$ 이면 $\Delta z = \Delta x$ 이므로 가능한 차분이고, $\Pr[\Delta x, 0 \rightarrow \Delta z] = \frac{1}{2}$ 이다.

역으로 $\Pr[\Delta x, 0 \rightarrow \Delta z] = \frac{1}{2}$ 이면 $w_h((\Delta x \vee \Delta z) \wedge \text{mask}(n-1)) = 1$,

즉 $i \in [1, n-2]$ 에서 $\Delta z_i = 0$ 이다. 그리고 가능한 차분의 조건으로 $i = 0$ 에서 $\Delta z_0 = \Delta x_0 = 1$ 이고, $i = n-1$ 에서 $\Delta x_{n-2} = \Delta z_{n-2} = 0$ ($n \geq 3$) $\Rightarrow \Delta z_{n-1} = \Delta x_{n-1} = 0$ 이므로 가능한 Δz 는 e_0 뿐이다.

또한, $\Delta z = e_0 \oplus e_1$ 은 $i = 0$ 에서 $\Delta z_0 = \Delta x_0 = 1$ 이고,

$i = 1$ 과 $i = 2$ 에서는 $\Delta x_0 = 1, \Delta z_1 = 1$ 로 0이 아니기 때문에 Δz_i 가 자유이며,

$i \geq 3$ 에서는 $\Delta x_{i-1} = \Delta z_{i-1} = 0$ 에 대해 $\Delta z_i = \Delta x_i = 0$ 이 성립하는 가능한 차분이다.

따라서 $w_h((\Delta x \vee \Delta z) \wedge \text{mask}(n-1)) = 2$ 이므로 $\Pr[\Delta x, 0 \rightarrow \Delta z] = \frac{1}{4}$ 이다.

$\Delta z = \Delta x$ 만 확률이 $\frac{1}{2}$ 이므로 그 외 모든 Δz 는 확률이 $\frac{1}{4}$ 이하이다. 따라서

$$\Pr[(\Delta x, \Delta y) \rightarrow \Delta z] = \begin{cases} \frac{1}{2}, & \Delta z = (0, \dots, 0, 1) = \Delta x, \\ \frac{1}{4}, & \Delta z = (0, \dots, 0, 1, 1), \\ \leq \frac{1}{4}, & \text{otherwise.} \end{cases}$$

Recovery of the outer affine maps - Proof of Proposition 4

② $\Delta x = \Delta x_2 = e_0 \oplus e_{n-2}$ 의 경우 ($n \geq 3$),

Δx 에 대하여 $n-2 \neq 0$ 이고 $\Delta x_0, \Delta x_{n-2}$ 가 모두 마스킹 범위 안에 있으므로

$w_h(\Delta x \wedge \text{mask}(n-1)) = 2$ 이고, 모든 Δz 에서 $\Pr[\Delta x, 0 \rightarrow \Delta z] \leq \frac{1}{4}$ 이다.

이때 $\Delta z = \Delta x_2 = (0, 1, \dots, 0, 1)$ 은 $i = 0$ 에서 $\Delta z_0 = \Delta x_0 = 1$ 이고,

$2 \leq i \leq n-2$ 에서 $\Delta x_{i-1} = \Delta z_{i-1} = 0$ 이면 $\Delta z_i = \Delta x_i$ 이므로 가능한 차분이다.

또한, $i = n-1$ 에서 $\Delta x_{n-1} = 1$ 이므로 Δz_{n-1} 은 제약이 없다. ($\Delta z = \Delta x$ 이므로 모든 i 에 대하여 가능한 차분 조건 성립)

따라서 $w_h((\Delta x \vee \Delta z) \wedge \text{mask}(n-1)) = 2$ 이므로 $\Pr[\Delta x, 0 \rightarrow \Delta z] = \frac{1}{4}$ 이다.

또한, $\Delta z = \Delta x \oplus e_{n-1} = (1, 1, \dots, 0, 1)$ 도 $i = n-1$ 에서 $\Delta x_{n-2} = 1 \neq 0$ 으로

$\Delta z_{n-1} = 1$ 에 제약이 없는 가능한 차분이다.

최상위 비트는 마스킹 범위 밖이므로 해밍무게는 그대로 2, $\Pr[\Delta x, 0 \rightarrow \Delta z] = \frac{1}{4}$ 이다.

모든 Δz 에서 확률이 $\frac{1}{4}$ 이하이고 위 두 경우가 이를 달성하므로,

$$\Pr[(\Delta x, \Delta y) \rightarrow \Delta z] = \begin{cases} \frac{1}{4}, & \Delta z = (0, 1, \dots, 0, 1) = \Delta x, \\ \frac{1}{4}, & \Delta z = (1, 1, \dots, 0, 1) = \Delta x \oplus e_{n-1}, \\ \leq \frac{1}{4}, & \text{otherwise.} \end{cases}$$

①, ②에 의해 Proposition 4가 증명된다.

□

Recovery of the outer affine maps

Proposition 4를 이용하여 T_1, T_2 를 복원하는 구체적인 절차는 Algorithm 3과 같다.

Algorithm 3 Recovering outer lower triangular matrices

Input: oracle $\mathbb{O}$ implementing bijective modular addition with partially recovered affine encodings (Subsection 6.1, Subsection 6.2)

Output: modified oracle $\mathbb{O}'$ that matches the structure from Figure 8 with T_1, T_2 being identity matrices with extra unknown coefficients in the bottom rows only.

```

1:  $\mathbb{O}_3 \leftarrow \mathbb{O}$ 
2: for  $w \in \{3, \dots, n-1\}$  do
3:    $M_1 \leftarrow$  identity  $(n-1) \times (n-1)$  matrix
4:    $M_2 \leftarrow$  identity  $(n-1) \times (n-1)$  matrix
5:   for  $i \in \{0, \dots, n-w-1\}$  do
6:      $\Delta x_1 \leftarrow e_{2n-w-i}$ 
7:      $\Delta x_2 \leftarrow e_{2n-w-i} \vee e_{2n-2-i}$ 
8:      $D_1 = \{(\Delta \mathbb{O}_w(\Delta x_1))_{2n-w-i, \dots, 2n-1-i} \mid \mathcal{O}(n) \text{ times}\}$   $\triangleright w$ -bit differences
9:      $D_2 = \{(\Delta \mathbb{O}_w(\Delta x_2))_{2n-w-i, \dots, 2n-1-i} \mid \mathcal{O}(n) \text{ times}\}$   $\triangleright w$ -bit differences
10:    if the most frequent difference  $\Delta_Z \in D_1$  occurred  $\approx 50\%$  times then
11:       $(M_1)_{n-2-i, n-w-i} \leftarrow 0$ 
12:       $(M_2)_{n-2-i, n-w-i} \leftarrow (\Delta_Z)_{w-2-i}$ 
13:    else if the most frequent difference  $\Delta_Z \in D_2$  occurred  $\approx 50\%$  times then
14:       $(M_1)_{n-2-i, n-w-i} \leftarrow 1$ 
15:       $(M_2)_{n-2-i, n-w-i} \leftarrow (\Delta_Z)_{w-2-i}$ 
16:    else
17:      retry this  $i$ 
18:    end if
19:  end for
20:   $\mathbb{O}_{w+1} \leftarrow \text{diag}(M_2, \text{Id}_{n-1}) \circ \mathbb{O}_w \circ \text{diag}(M_1, \text{Id}_{n-1})$ 
21:   $\triangleright \text{diag}(M, \text{Id}_{n-1})$  extends the matrix  $M$  to  $N \times N$  matrix by an identity matrix
22: end for
23: return  $\mathbb{O}' = \mathbb{O}_n$ 

```

Recovery of the outer affine maps - Diagonals Recovery

Line 6에서 $\Delta x_1 = e_{2n-w-i}$ 은 window 내에서 e_0 에 해당하며,

Line 7에서 $\Delta x_2 = e_{2n-w-i} \oplus e_{2n-2-i}$ 은 window 내에서 e_{w-2} 에 해당함

임의의 벡터 $u = (u_4 \ u_3 \ u_2 \ u_1 \ u_0) \in \mathbb{F}_2^5$ 와 단위벡터 e_i 에 대하여 다음과 같다.

$$\begin{pmatrix} 1 & & & & \\ & 1 & & & \\ & & 1 & & \\ m_1 & & & 1 & \leftarrow i \\ & & & & 1 \end{pmatrix} \begin{pmatrix} u_4 \\ u_3 \\ u_2 \\ u_1 \\ u_0 \end{pmatrix} = \begin{pmatrix} u_4 \\ u_3 \\ u_2 \\ u_1 \oplus m_1 u_3 \\ u_0 \end{pmatrix}$$

$$= u \oplus (m_1 u_3) e_3$$

이때, m_1 의 index는 $(n-2)-i, n-2-(w-2)-i$ 로,

$(M_1)_{(n-2)-i, n-2-(w-2)-i}$ 에 해당한다.

이를 일반화하면, $m_1 := (M_1)_{(n-2)-i, n-2-(w-2)-i} \in \mathbb{F}_2$ 에

대하여, 임의의 $u \in \mathbb{F}_2^{n-1}$ 은 다음을 만족한다.

$$M_1 u = u \oplus m_1 u_{n-w-i} e_{w-2}$$

즉,

$$M_1 \Delta x_1 = \begin{cases} \Delta x_1, & \text{if } m_1 = 0 \\ \Delta x_2, & \text{if } m_1 = 1 \end{cases}, \quad M_1 \Delta x_2 = \begin{cases} \Delta x_2, & \text{if } m_1 = 0 \\ \Delta x_1, & \text{if } m_1 = 1 \end{cases}$$

u 의 index $n-w-i$ 는 $n-1$ bit 중 아래부터 w 크기의 윈도우를 i 씩 위로 올릴 때 윈도우에서 $w-2$ 번째 비트로 $w-2-i$ 로 볼 때 이해하기 쉬울 수 있음

Algorithm 3 Recovering outer lower triangular matrices

Input: oracle $\mathbb{O}$ implementing bijective modular addition with partially recovered affine encodings (Subsection 6.1, Subsection 6.2)

Output: modified oracle $\mathbb{O}'$ that matches the structure from Figure 8 with T_1, T_2 being identity matrices with extra unknown coefficients in the bottom rows only.

```

1:  $\mathbb{O}_3 \leftarrow \mathbb{O}$ 
2: for  $w \in \{3, \dots, n-1\}$  do
3:    $M_1 \leftarrow$  identity  $(n-1) \times (n-1)$  matrix
4:    $M_2 \leftarrow$  identity  $(n-1) \times (n-1)$  matrix
5:   for  $i \in \{0, \dots, n-w-1\}$  do
6:      $\Delta x_1 \leftarrow e_{2n-w-i} = e_0$ 
7:      $\Delta x_2 \leftarrow e_{2n-w-i} \vee e_{2n-2-i} = e_{w-2}$ 
8:      $D_1 = \{(\Delta \mathbb{O}_w(\Delta x_1))_{2n-w-i, \dots, 2n-1-i} \mid \mathcal{O}(n) \text{ times}\}$   $\triangleright w$ -bit differences
9:      $D_2 = \{(\Delta \mathbb{O}_w(\Delta x_2))_{2n-w-i, \dots, 2n-1-i} \mid \mathcal{O}(n) \text{ times}\}$   $\triangleright w$ -bit differences
10:    if the most frequent difference  $\Delta_Z \in D_1$  occurred  $\approx 50\%$  times then
11:       $(M_1)_{n-2-i, n-w-i} \leftarrow 0$ 
12:       $(M_2)_{n-2-i, n-w-i} \leftarrow (\Delta_Z)_{w-2-i}$ 
13:    else if the most frequent difference  $\Delta_Z \in D_2$  occurred  $\approx 50\%$  times then
14:       $(M_1)_{n-2-i, n-w-i} \leftarrow 1$ 
15:       $(M_2)_{n-2-i, n-w-i} \leftarrow (\Delta_Z)_{w-2-i}$ 
16:    else
17:      retry this  $i$ 
18:    end if
19:  end for
20:   $\mathbb{O}_{w+1} \leftarrow \text{diag}(M_2, \text{Id}_{n-1}) \circ \mathbb{O}_w \circ \text{diag}(M_1, \text{Id}_{n-1})$ 
21:   $\triangleright \text{diag}(M, \text{Id}_{n-1})$  extends the matrix  $M$  to  $N \times N$  matrix by an identity matrix
22: end for
23: return  $\mathbb{O}' = \mathbb{O}_n$ 

```

Recovery of the outer affine maps - Diagonals Recovery

- Δx_1 에 대하여 w 윈도우 내에서 최대차분확률이 $\frac{1}{2}$ 이면 T_1 대상 비트 위치 $(n-2-i, n-w-i)$ 값을 건드릴 필요가 없다.
- Δx_2 에 대하여 윈도우 내에서 차분 확률이 $\frac{1}{2}$ 이면 T_1 대상 비트 위치 값에 의해 Δx_2 의 윈도우 범위 비트열이 Δx_1 로 바뀌었다고 보고 M_1 으로 보정해준다.

어떤 w 에서 결정된 $(M_1)_{(n-2)-i, n-2-(w-2)-i}$ 들에 의해 영향을 받은 다른 행-열의 비트들은 다음 w 단계에서 보정해준다.

- ① 먼저 Δx_1 로 샘플링한 출력 차분의 약 $1/2$ 이 특정 Δz 로 구성된 경우,
 (Line 11) T_1 은 Δx_1 을 통과시켰다고 보고,
 (Line 12) T_2 는 Δz 만큼 변환되었다고 봄

```

6:       $\Delta x_1 \leftarrow e_{2n-w-i}$ 
8:       $D_1 = \{(\Delta \mathbb{O}_w(\Delta x_1))_{2n-w-i, \dots, 2n-1-i} \mid \mathcal{O}(n) \text{ times}\}$  ▷  $w$ -bit differences
10:     if the most frequent difference  $\Delta_Z \in D_1$  occurred  $\approx 50\%$  times then
11:          $(M_1)_{n-2-i, n-w-i} \leftarrow 0$ 
12:          $(M_2)_{n-2-i, n-w-i} \leftarrow (\Delta_Z)_{w-2-i}$ 

```

- ② ①을 만족하지 않는 때, Δx_2 로 샘플링한 출력 차분의 약 $1/2$ 이 특정 Δz 로 구성된 경우,
 (Line 14) T_1 은 Δx_2 를 Δx_1 으로 변형시킨 채 통과시켰다고 보고,
 (Line 15) T_2 는 Δz 만큼 변환되었다고 봄

```

7:       $\Delta x_2 \leftarrow e_{2n-w-i} \vee e_{2n-2-i}$ 
9:       $D_2 = \{(\Delta \mathbb{O}_w(\Delta x_2))_{2n-w-i, \dots, 2n-1-i} \mid \mathcal{O}(n) \text{ times}\}$  ▷  $w$ -bit differences
13:     else if the most frequent difference  $\Delta_Z \in D_2$  occurred  $\approx 50\%$  times then
14:          $(M_1)_{n-2-i, n-w-i} \leftarrow 1$ 
15:          $(M_2)_{n-2-i, n-w-i} \leftarrow (\Delta_Z)_{w-2-i}$ 

```

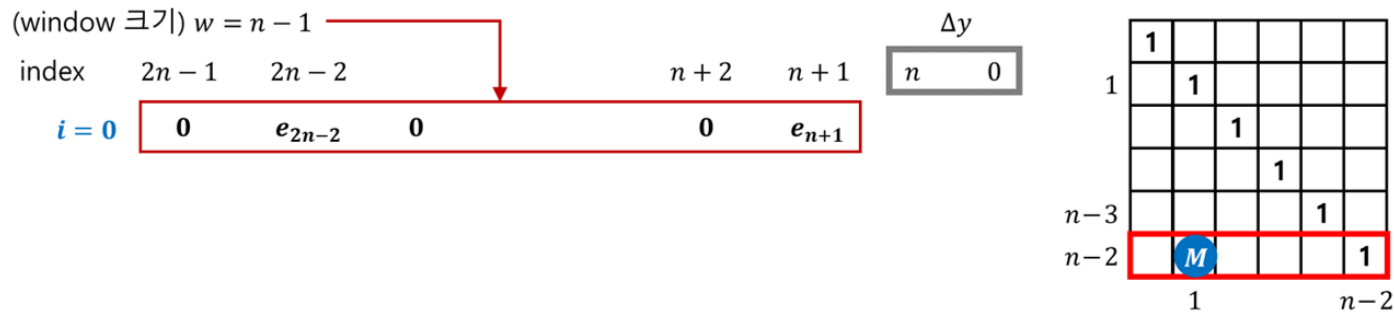
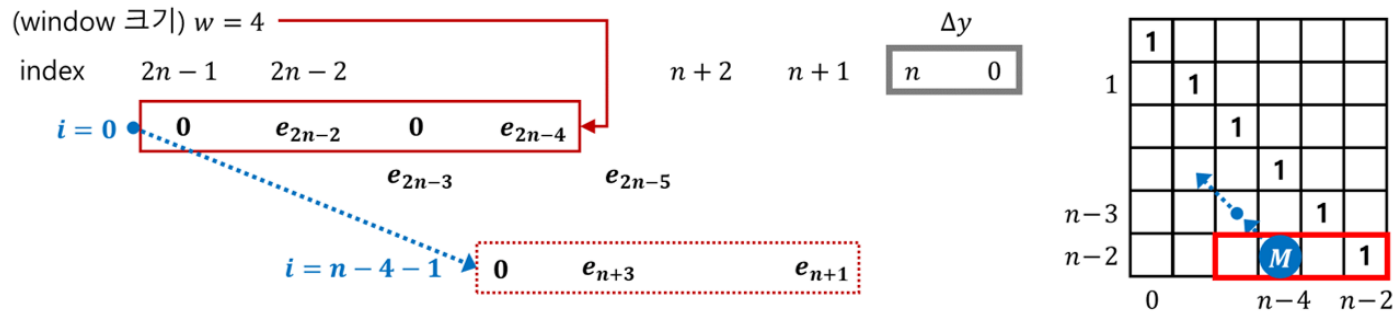
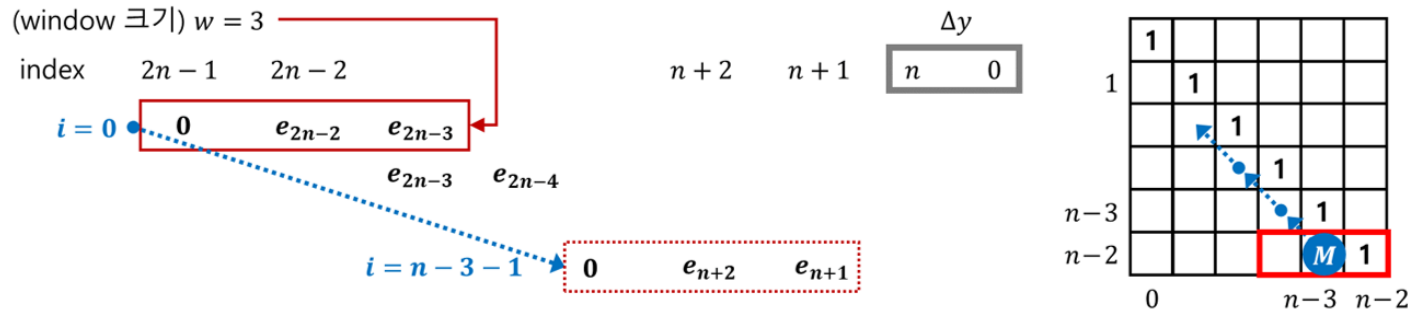
- ③ 이를 반영하여 오라클을 갱신함

```

20:    $\mathbb{O}_{w+1} \leftarrow \text{diag}(M_2, \text{Id}_{n-1}) \circ \mathbb{O}_w \circ \text{diag}(M_1, \text{Id}_{n-1})$ 

```

Recovery of the outer affine maps - Diagonals Recovery



Recovery of the outer affine maps - Bottom Row Recovery

Algorithm 3을 수행하고 나면 남아있는 행에 대한 복원이 필요하다.

이때, T_1 의 남은 행은 차분의 선형성에 의해 임의로 설정 가능하다.

n 비트 모듈러 덧셈 $z = x \boxplus y$ 에서 최상위 비트는

$$z_{n-1} = x_{n-1} \oplus y_{n-1} \oplus c_{n-1}$$

이고, $c_{n-1} = \text{maj}(x_{n-2}, y_{n-2}, c_{n-2})$ 는 x_{n-1} 에 의존하지 않는다.

x_{n-1} 에만 의존하여 차분 변화가 발생할 때, $\Delta z_{n-1} = \Delta x_{n-1}$ 이 항상 성립한다.

이는 다음과 같은 자기동치를 만든다.

$$t_\alpha \circ S \circ t_\alpha = S \quad \text{where} \quad t_\alpha : x \mapsto x \oplus \langle \alpha, x \rangle e_{N-1} \quad \text{for some } \alpha \in \mathbb{F}_2^N, \alpha_{N-1} = 0.$$

Proof) 먼저, $x, \alpha \in \mathbb{F}_2^N$ 에 대하여 $\alpha_{N-1} = 0$ 이므로

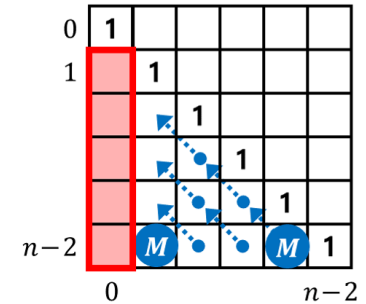
$$\langle \alpha, x \oplus \langle \alpha, x \rangle e_{N-1} \rangle = \langle \alpha, x \rangle \oplus \langle \alpha, \langle \alpha, x \rangle e_{N-1} \rangle = \langle \alpha, x \rangle \oplus \langle \alpha, x \rangle \langle \alpha, e_{N-1} \rangle = \langle \alpha, x \rangle$$

이고,

$$\begin{aligned} t_\alpha(t_\alpha(x)) &= (x \oplus \langle \alpha, x \rangle e_{N-1}) \oplus \langle \alpha, (x \oplus \langle \alpha, x \rangle e_{N-1}) \rangle e_{N-1} \\ &= x \oplus \langle \alpha, x \rangle e_{N-1} \oplus \langle \alpha, x \rangle e_{N-1} \\ &= x \end{aligned}$$

$$\implies t_\alpha \circ t_\alpha = I$$

이다.



Recovery of the outer affine maps

한편

$$\begin{aligned}
 (x \boxplus y) \oplus \langle \alpha, x \rangle e_{N-1} &= (x_{N-1} \oplus y_{N-1} \oplus c_{N-1}, x_{N-2} \oplus y_{N-2} \oplus c_{N-2}, \dots, x_0 \oplus y_0 \oplus c_0) \oplus (\langle \alpha, x \rangle, 0, \dots, 0) \\
 &= ((x_{N-1} \oplus \langle \alpha, x \rangle) \oplus y_{N-1} \oplus c_{N-1}, (x_{N-2} \oplus 0) \oplus y_{N-2} \oplus c_{N-2}, \dots, (x_0 \oplus 0) \oplus y_0 \oplus c_0) \\
 &= (x \oplus \langle \alpha, x \rangle e_{N-1}) \boxplus y
 \end{aligned}$$

이므로

$$t_\alpha \circ S(x) = S \circ t_\alpha(x) \implies t_\alpha \circ S \circ t_\alpha = S \circ t_\alpha \circ t_\alpha.$$

따라서 $t_\alpha \circ S \circ t_\alpha = S$ 이다. □

즉, 모듈러 덧셈의 입력의 최상위 비트에 XOR되는 선형항 $\langle \alpha, x \rangle$ 는 S 의 출력에서도 오직 최상위 비트에만 나타난다.

따라서 공격의 편의를 위하여 최상위 비트에 관한 T_1 의 남아있는 행을 영벡터로 고정한다.

다음으로 결정한 T_1 에 대한 T_2 를 구한다.

T_2 역시 Algorithm 30이 종료된 후 최상위 비트에 대한 행만 복구되지 않은 상황이다.

T_2 의 남은 행을 구하기 위해 Lemma 2를 사용한다.

Recovery of the outer affine maps

Lemma 2. Let $(\Delta e_i, 0) \xrightarrow{\boxplus} \Delta z$ be a differential transition through the n -bit addition, $i \leq n - 2$. Then, $(\Delta z)_{i+1} = 0$ implies that $\Delta z = e_i$.

Proof) 증명에 앞서 $\Delta z_j = \Delta x_j \oplus \Delta c_j$ 이고, $\text{maj}(a, b, c) = ab \oplus bc \oplus ca = a(b \oplus c) \oplus bc$ 이므로 다음이 성립한다.

$$\text{maj}(a, b, c) \oplus \text{maj}(a \oplus 1, b, c) = (a(b \oplus c) \oplus bc) \oplus ((a \oplus 1)(b \oplus c) \oplus bc) = b \oplus c$$

- ① $j \leq i$ 에 대하여, $\Delta c_j = 0$ 이므로, $\Delta z_j = \Delta x_j = 0$ 이다.
- ② $j = i$ 에 대하여, $\Delta c_i = 0$ 이므로, $\Delta z_i = \Delta x_i = 1$ 이다.
- ③ $j = i + 1$ 에 대하여, i 번째 carry bit는 $i - 1$ 번째 비트들의 영향을 받으므로 $c'_i = c_i$ 이고, Lemma의 가정에 따라 $x'_i = x_i \oplus 1$ 이므로

$$\Delta c_{i+1} = \text{maj}(x_i, y_i, c_i) \oplus \text{maj}(x_i \oplus 1, y_i, c_i) = y_i \oplus c_i$$

이다. 또한, $\Delta x_{i+1} = 0$ 이므로

$$(\Delta z)_{i+1} = \Delta x_{i+1} \oplus \Delta c_{i+1} = y_i \oplus c_i$$

이다. (여기에서 $i + 1$ 이 $n - 1$ 비트 워드에 들어와야 하므로 $i \leq n - 2$ 라는 조건이 주어짐)

이때 $(\Delta z)_{i+1} = 0$ 이면, $y_i \oplus c_i = 0 \implies \Delta c_{i+1} = 0$ 이므로 $c'_{i+1} = c_{i+1}$ 이다.

- ④ $j \geq i + 1$ 에 대하여, $x_j = x'_j$ 이고, $c_{i+1} = c'_{i+1}$ 이므로, 모든 j 에 대해 $c_j = c'_j$ 이다. 따라서

$$\Delta z_j = \Delta x_j \oplus \Delta c_j = 0.$$

결과적으로, $\Delta z_i = 1$ 이고 $j \neq i$ 인 모든 j 에 대하여 $\Delta z_j = 0$ 이므로 $\Delta z = e_i$ 이다. □

Recovery of the outer affine maps

T_1 을 완전히 복구한 상황이므로,

Δe_j ($j \in [0, n-4]$)에 대한 입력을 반복하여 T_2 의 남아있는 행을 다음과 같이 직접 복구할 수 있다.

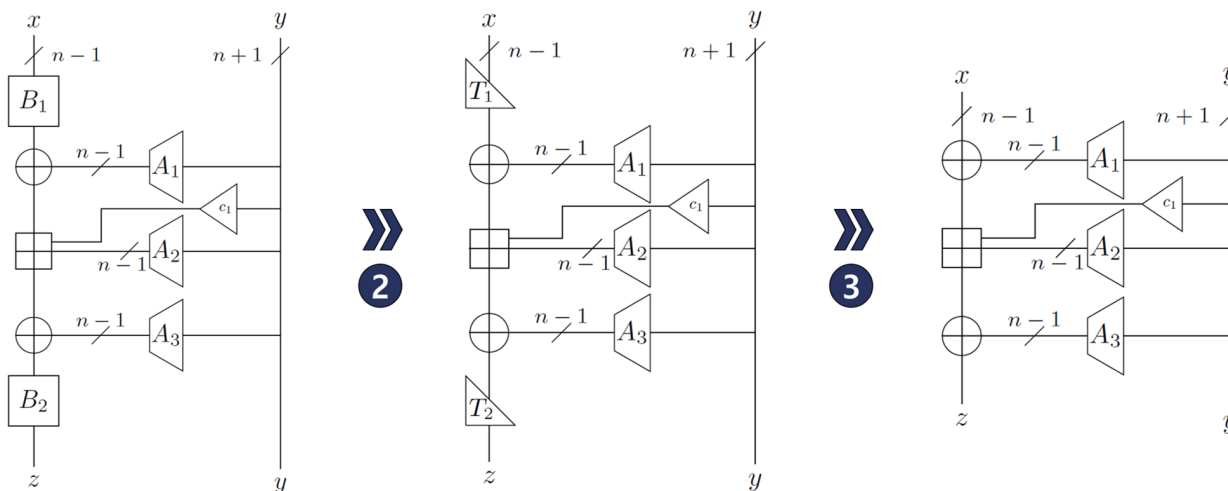
- ❶ 같은 j 에 대해 입력 차분 e_j 를 여러 번 입력하여 출력 차분 Δz 를 모은다.
- ❷ 그중 $(\Delta z)_{j+1} = 1$ 인 샘플은 그냥 버린다. (원하는 조건에 맞지 않음)
- ❸ $(\Delta z)_{j+1} = 0$ 인 샘플만 모아서 해당하는 비트를 구한다.

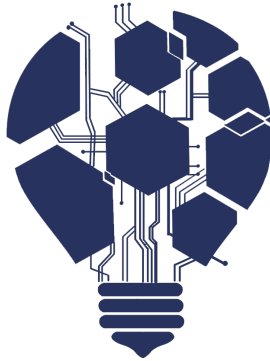
단, $j = n-3$ 일 때는 비트를 구할 수 없다.

Lemma 2의 조건은 $i \leq n-3$ (Lemma 2는 n 비트 덧셈을 기준으로 하며, 현재 공격은 $n-1$ 비트를 기준으로 수행하고 있음)이지만, $(\Delta z)_{j+1} = 0$ 가 되는지 확인할 때 필요한 $(\Delta z)_{n-2}$ 를 사용할 수 없기 때문이다.

따라서 해당 비트는 임의로 설정하고, 이후 수행되는 단계 4에서 보정한다.

단계 3을 수행하면 남은 구조는 다음과 같다.



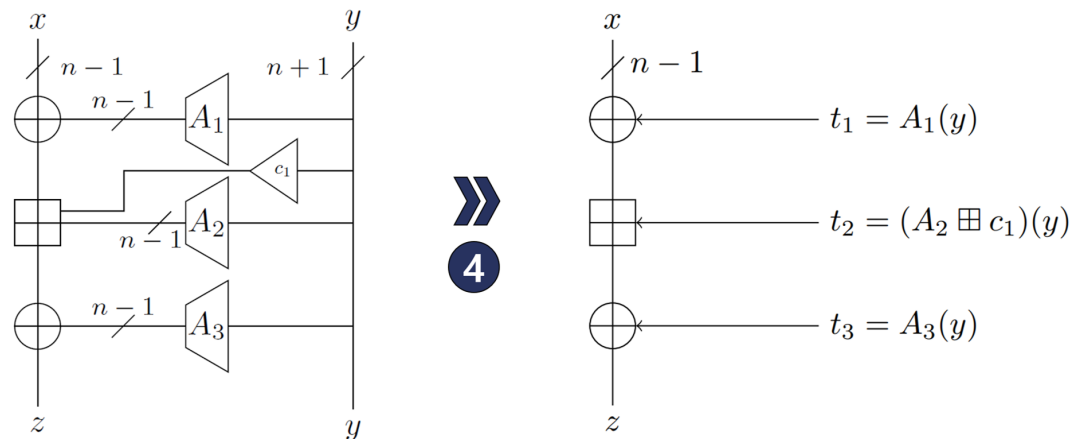


- ① 1: Locating Linear Bits
- ② 2: Triangularization of the outer affine maps
- ③ 3: Recovery of the outer affine maps (left branches)
- ④ 4: Recovery of the Feistel affine maps (right to left XOR and addition)
- ⑤ 5: Correction of the first carry bit

Recovery of the Feistel affine maps

단계 4의 공격 목표는 남아있는 affine map A_1, A_2, A_3 를 제거하여 라운드 함수에 존재하는 미지의 affine map이 없도록 만드는 것이다.

남아있는 Feistel affine map의 right branch를 random y 로 고정하면, $A_1(y), A_2(y), A_3(y)$ 는 상수 $t_1, t_2, t_3 \in \mathbb{F}_2^{n-1}$ 로 고정된다.



$\{(t_1, t_2, t_3)\}$ 후보는 $2^{3(n-1)}$ 개로 전수조사 불가하므로, 모듈러 덧셈의 특성을 활용하여 LSB부터 $(t_1)_j, (t_2)_j, (t_3)_j$ 한 비트씩 복구한다.

Recovery of the Feistel affine maps

Proposition 5는 $(t_2)_0 = 1$ 일 때, 특정 y 에 대한 t_1, t_2, t_3 의 후보를 (실험적으로) 8개의 후보로 줄인다.

Proposition 5. Let $n \geq 2, t_1, t_2, t_3 \in \mathbb{F}_2^n$ with $(t_2)_0 = 1$. Then, all the maps $(\oplus t'_3) \circ (\boxplus t'_2) \circ (\oplus t'_1)$ are equivalent to the map $(\oplus t_3) \circ (\boxplus t_2) \circ (\oplus t_1)$ with

$$t'_1 = t_1 \oplus a_1 \cdot (1, 0, \dots, 0) \oplus a_3 \cdot (1, \dots, 1),$$

$$t'_2 = t_2 \oplus (a_1 \oplus a_2) \cdot (1, 0, \dots, 0) \oplus a_3 \cdot (1, \dots, 1, 0),$$

$$t'_3 = t_3 \oplus a_2 \cdot (1, 0, \dots, 0) \oplus a_3 \cdot (1, \dots, 1),$$

for any $a_1, a_2, a_3 \in \mathbb{F}_2$. In particular, there are 8 equivalent triples, whenever the least significant bit of t_2 is 1.

Proof) 모듈러 덧셈의 MSB는 차분에 대하여 선형이므로, 임의의 t'_1, t'_2, t'_3 에 대한 $\text{map } (\oplus t'_3) \circ (\boxplus t'_2) \circ (\oplus t'_1)$ 가 다음을 만족하는 t_1, t_2, t_3 에 대한 $\text{map } (\oplus t_3) \circ (\boxplus t_2) \circ (\oplus t_1)$ 과 동치이다.

$$t'_1 = t_1 \oplus a_1 \cdot (1, 0, \dots, 0), \quad t'_2 = t_2 \oplus (a_1 \oplus a_2) \cdot (1, 0, \dots, 0), \quad t'_3 = t_3 \oplus a_2 \cdot (1, 0, \dots, 0), \quad \forall a_1, a_2 \in \mathbb{F}_2.$$

$$\begin{aligned} (\oplus t'_3) \circ (\boxplus t'_2) \circ (\oplus t'_1) &= (\oplus ((t_3)_{n-2} \oplus a_2, (t_3)_{n-3}, \dots, (t_3)_0)) \\ &\quad \circ (\boxplus ((t_2)_{n-2} \oplus a_1 \oplus a_2, (t_2)_{n-3}, \dots, (t_2)_0)) \circ (\oplus ((t_1)_{n-2} \oplus a_1, (t_1)_{n-3}, \dots, (t_1)_0)) \\ &= (\oplus ((t_3)_{n-2}, (t_3)_{n-3}, \dots, (t_3)_0)) \\ &\quad \circ (\boxplus ((t_2)_{n-2}, (t_2)_{n-3}, \dots, (t_2)_0)) \circ (\oplus ((t_1)_{n-2}, (t_1)_{n-3}, \dots, (t_1)_0)) \\ &= (\oplus t_3) \circ (\boxplus t_2) \circ (\oplus t_1). \end{aligned}$$

Recovery of the Feistel affine maps

따라서 $a_1 = a_2 = 0$ 인 경우만 고려하며, 이제 $a_3 = 1$ 일 때를 확인한다.

$$t'_1 = t_1 \oplus a_3 \cdot (1, \dots, 1), \quad t'_2 = t_2 \oplus a_3 \cdot (1, \dots, 1, 0), \quad t'_3 = t_3 \oplus a_3 \cdot (1, \dots, 1),$$

임의의 $x \in \mathbb{F}_2$ 에 대하여, $z = ((\oplus t_3) \circ (\boxplus t_2) \circ (\oplus t_1))(x)$ 라고 하자. 즉,

$$z = [(x \oplus t_1) \boxplus t_2] \oplus t_3 \Leftrightarrow (z \oplus t_3) = (x \oplus t_1) \boxplus t_2$$

이다. 다음 식

$$x \oplus (1, \dots, 1) = \neg x = \boxminus x \boxminus 1 \quad \text{and} \quad \neg(x \oplus y) = x \oplus \neg y$$

를 사용하면 위 식을

$$\begin{aligned} z \oplus \neg t_3 &= \neg(z \oplus t_3) = \boxminus(z \oplus t_3) \boxminus 1 \\ &= \boxminus((x \oplus t_1) \boxplus t_2) \boxminus 1 \\ &= \boxminus(x \oplus t_1) \boxminus t_2 \boxminus 1 \\ &= \boxminus(x \oplus t_1) \boxminus 1 \boxplus 1 \boxplus (\boxminus t_2 \boxminus 1) \boxplus 1 \boxminus 1 \\ &= \neg(x \oplus t_1) \boxplus 1 \boxplus \neg t_2 \boxplus 1 \boxminus 1 \\ &= (x \oplus \neg t_1) \boxplus (\neg t_2 \boxplus 1) \end{aligned}$$

와 같이 정리할 수 있다.

Recovery of the Feistel affine maps

이때 t_2 의 LSB가 1이므로, $\neg t_2$ 의 LSB는 0이고, 따라서

$$(\neg t_2 \boxplus 1) = \neg t_2 \oplus 1 = t_2 \oplus (1, \dots, 1, 0)$$

이다. 결과적으로

$$\begin{aligned} z &= [(x \oplus \neg t_1) \boxplus (t_2 \oplus (1, \dots, 1, 0))] \oplus \neg t_3 \\ &= [(x \oplus t_1 \oplus (1, \dots, 1)) \boxplus (t_2 \oplus (1, \dots, 1, 0))] \oplus (t_3 \oplus (1, \dots, 1)) \\ &= [(x \oplus t'_1) \boxplus t'_2] \oplus t'_3 \end{aligned}$$

이고, 정리하면 $[(x \oplus t_1) \boxplus t_2] \oplus t_3 = [(x \oplus t'_1) \boxplus t'_2] \oplus t'_3$ 로 두 매핑은 서로 같다.

※ 논문에서는 증명이 여기까지 제시되어 있고, $t_2 = 1$ 일 때 "8개"의 a_1, a_2, a_3 쌍을 가진다는 것은 실험적으로 확인하고 있음. □

이때 임의의 y 에 대하여 $(t_2)_0 = 1$ 일 확률은 $\frac{1}{2}$ 이므로, y 를 평균 2번의 시도로 고정할 수 있다.

후보가 9개 이상 남을 경우 다른 y 를 선택

Proposition 5는 공격자가 내부를 확인할 수 없는 오라클을 공격할 때

입력한 y 에 대하여 본질적으로 구별할 수 없는 동치 후보가 8개 존재함을 보여준다.

공격자는 비트를 하나씩 늘려가며 $\text{mod } 2, \text{mod } 4, \dots, \text{mod } 2^{n-1}$ 순으로 bit-by-bit search를 수행한다.

비트마다 가능한 후보 8개에 대하여 다음 비트에서 구한 후보와 비교하여 조건이 맞지 않는 후보를 제거한다.

[1]은 실험적으로 각 비트 단계에서 최대 8개의 후보가 유지됨을 확인하였다.

후보가 8개를 초과하면 다른 y 를 선택한다.

Recovery of the Feistel affine maps

A_1, A_2, A_3 가 $\mathbb{F}_2^{n+1} \rightarrow \mathbb{F}_2^{n-1}$ affine map이므로

$n + 2$ 개의 y 를 사용하여 $\{(t_1, t_2, t_3)\}$ 를 구하면 $A_1, A_2 \boxplus c_1, A_3$ 를 복구할 수 있다.

단 $t_2 = (A_2 \boxplus c_1)(y)$ 를 하나의 Affine map으로 보고 구한 것이므로

A_2 와 $c_1(y)$ 에 대한 분리를 추가로 수행해야 한다. $c_1(y) \in \{0, 1\}$ 이므로,

$$t_2(y) = A_2(y) \boxplus c_1(y) = \begin{cases} A_2(y) + 1 \pmod{2^{n-1}} & \text{if } c_1(y) = 1, \\ A_2(y) & \text{if } c_1(y) = 0 \end{cases}$$

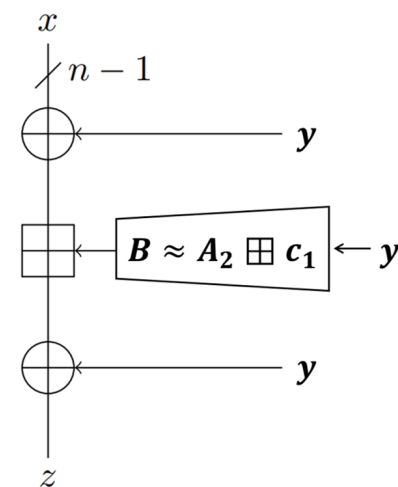
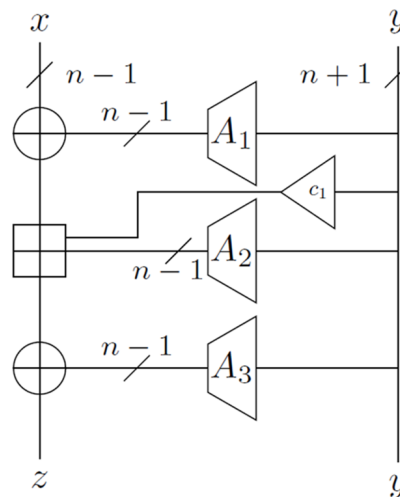
이다. $B \approx A_2 \boxplus c_1$ 이라고 할 때, c_1 를 교정하는 c' 은 단계 5에서 구해준다.

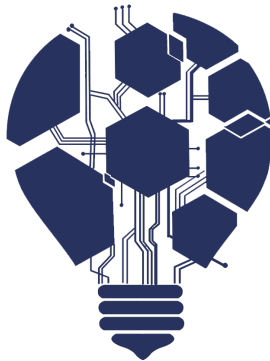
단계 4가 완료된 후 남아있는 오라클 구조는 다음과 같다.

$$O(x, y) = (x \boxplus B(y) \boxplus c'(y), y)$$

where $c'(y) \in \{-1, 0, 1\}$.

연산량 추가

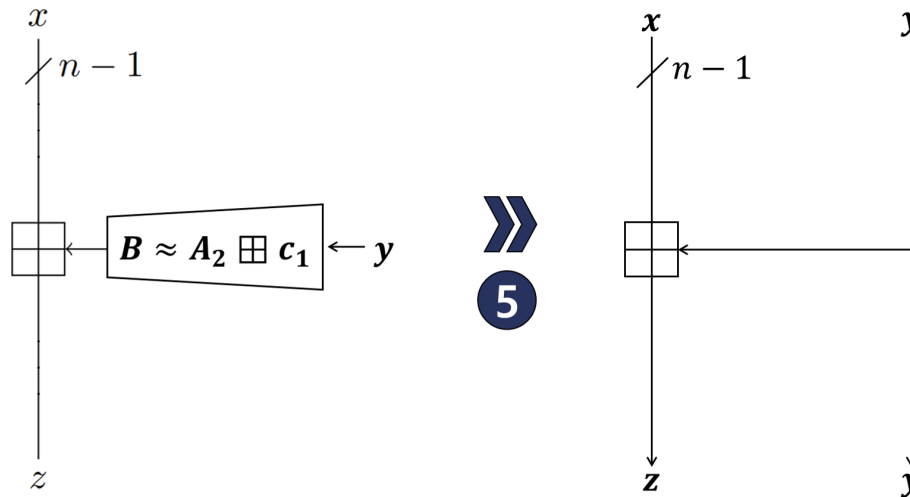




- ① 1: Locating Linear Bits
- ② 2: Triangularization of the outer affine maps
- ③ 3: Recovery of the outer affine maps (left branches)
- ④ 4: Recovery of the Feistel affine maps (right to left XOR and addition)
- ⑤ 5: Correction of the first carry bit

Recovery of the Feistel affine maps

단계 4에서 복구한 affine map 중 A_2 는 carry bit c_1 를 고려하지 않고 복구한 것이므로, c_1 에 대한 정보와 분리되어야 한다.



따라서 단계 5에서는 단계 4에서 복구한 $A_2 \boxplus c_1$ 을 $B(y) = A_2(y) \boxplus c_1(y)$ 로 정의하고,

$$\mathbb{O}(x, y) = (x \boxplus B(y) \boxplus c'(y), y) \quad \text{where} \quad c'(y) \in \{-1, 0, 1\}$$

의 c' 을 복구한다.

Recovery of the Feistel affine maps

이때, 공격자는 위 모듈러 덧셈이

$$\begin{array}{c|cccccc|c}
 & x_{n-1} & x_{n-2} & \cdots & x_1 & x_0 & \\
 y_n \boxplus & y_{n-1} & y_{n-2} & \cdots & y_2 & y_1 & y_0 \\
 \hline
 = x_0 \oplus y_0 & x_{n-1} \oplus y_{n-1} \oplus c_{n-1} & x_{n-2} \oplus y_{n-2} \oplus c_{n-2} & \cdots & x_2 \oplus y_2 \oplus c_2 & x_1 \oplus y_1 \oplus x_0 y_0 & y_0 \\
 & = x_{n-2} \oplus y_{n-1} \oplus c_{n-1} & = x_{n-3} \oplus y_{n-2} \oplus c_{n-2} & & = x_1 \oplus y_2 \oplus c_2 & = x_0 \oplus y_1 \oplus c_1 &
 \end{array}$$

의 결과이므로

$$(x, y) \mapsto (x \boxplus y_{[1:n]} \boxplus (y_n \oplus y_0)y_0, y)$$

와 같다는 것을 알고 있다. 표기의 편의를 위하여 $y_{[1:n]} = (y_{n-1}, y_{n-2}, \dots, y_2, y_1)$

$$\begin{aligned}
 & (y_n, y_{n-1}, y_{n-2}, \dots, y_2, y_1, y_0) \\
 & \mapsto (y'_n, y'_{n-1}, y'_{n-2}, \dots, y'_2, y'_1, y'_0) = (y_{n-1}, y_{n-2}, \dots, y_2, y_1, y_0, y_n \oplus y_0)
 \end{aligned}$$

와 같이 인덱스가 조정된 입력 y 를 사용한다고 하면

$$(x, y) \mapsto (x \boxplus y_{[2:n+1]} \boxplus y_1 y_0, y)$$

가 된다. 이 식을 공격 대상인

$$\mathbb{O}(x, y) = (x \boxplus B(y) \boxplus c'(y), y) \quad \text{where } c'(y) \in \{-1, 0, 1\}$$

와 일치시키려면 B 에 대한 역함수를 구할 수 있어야 하는데,

$B : \mathbb{F}_2^{n+1} \rightarrow \mathbb{F}_2^{n+1}$ 이므로 B 의 차원에 대한 조정이 필요하다.

Recovery of the Feistel affine maps

$B : (n - 1) \times (n + 1)$ 행렬이므로 B 가 full-rank matrix라면($\text{rank } B = n - 1$) full-rank가 아니면 단계 4에서 다른 해 선택,
 B 에 임의의 독립인 행 2개($\tilde{B}$)를 추가해서 $B' : (n + 1) \times (n + 1)$ full-rank 가역 행렬을 만들 수 있다. 그렇다면

$$\pi_x(x, y) = (x, B'(y)), \quad \pi_y(x, y) = (x, B'(y))$$

를 사용하여 새로운 오라클 $\mathbb{O}' = \pi_y \circ \mathbb{O} \circ \pi_x^{-1}$ 을 정의할 수 있다.

$$\pi_x^{-1}(x, y) = (x, (B')^{-1}(y)) \text{ 이므로}$$

$$\begin{aligned} \mathbb{O}'(x, y) &= \pi_y \circ \mathbb{O} \circ \pi_x^{-1}(x, y) = \pi_y(\mathbb{O}(x, (B')^{-1}(y))) \\ &= \pi_y(x \boxplus B((B')^{-1}(y)) \boxplus c'((B')^{-1}(y)), (B')^{-1}(y)) \\ &= (x \boxplus B((B')^{-1}(y)) \boxplus c'((B')^{-1}(y)), B'((B')^{-1}(y))) \end{aligned}$$

이다. 이때 $B' = \begin{pmatrix} B \\ \tilde{B} \end{pmatrix}$ 이고

$$(B' \circ (B')^{-1})(y) = \begin{pmatrix} B \\ \tilde{B} \end{pmatrix} (B')^{-1}(y) = \begin{pmatrix} B(B')^{-1} \\ \tilde{B}(B')^{-1} \end{pmatrix} (y) = y$$

이므로

$$B(B')^{-1}(y) = (y_n, y_{n-1}, \dots, y_2)^T$$

이다.

Recovery of the Feistel affine maps

따라서 구하려던 대상

$$\begin{aligned}\mathbb{O}'(x, y) &= (x \boxplus B((B')^{-1}(y)) \boxplus c'((B')^{-1}(y)), y) \quad \text{where } c'(y) \in \{-1, 0, 1\} \\ &= (x \boxplus y_{[2:n+1]} \boxplus y_1 y_0, y)\end{aligned}$$

과 비교할 때

$$c'((B')^{-1}(y)) = y_1 y_0$$

를 구하면 된다.

단계 5 시작에서 처음 복구하려던 $c_1(y)$ 는 0 또는 1이므로, 먼저 $c' \in \{-1, 0, 1\}$ 에서 -1 을 없애야 한다.
 c_1 함수와 B' 모두 affine map이므로 -1 을 식별하는 문제는

$$c'((B')^{-1}(y)) = -1 \iff \tau(y) = 1$$

을 만족하는 특정 affine map $\tau : \mathbb{F}_2^{n+1} \rightarrow \mathbb{F}_2$ 를 구하는 문제가 된다.

$c'((B')^{-1}(y))$ 는 오라클 질의 1회로 구할 수 있다.

$$\begin{aligned}(z, y) &= \mathbb{O}'(0, y) = ((y_n, \dots, y_2) \boxplus c'((B')^{-1}(y)), y) \\ \therefore c'((B')^{-1}(y)) &= z \boxminus (y_n, \dots, y_2) \pmod{2^{n-1}}\end{aligned}$$

단, $c' \in \{-1, 0, 1\}$ 이므로 $2^{n-1} - 1 = -1$ 로 본다.

Recovery of the Feistel affine maps

τ 를 구하기 위하여 임의의 affine map

$$\tau(y) = a^T y + b \quad (a \in \mathbb{F}_2^{n+1}, b \in \mathbb{F}_2)$$

를 정의하면, τ 의 $(n+1) + 1$ 개 미지수를 구하기 위해 $n+2$ 개 y 에 대한 선형방정식

$$\begin{cases} a^T y^{(1)} + b = [c'((B')^{-1}(y^{(1)})) = -1] \\ \vdots \\ a^T y^{(n+2)} + b = [c'((B')^{-1}(y^{(n+2)})) = -1] \end{cases}$$

을 풀 수 있다.

이렇게 정의한 τ 는 $c'((B')^{-1}(y)) = -1$ 일 때만 $\tau(y) = 1$ 이므로 $\tau(y) = 0 \iff c'((B')^{-1}(y)) \neq -1$,
 y 의 최하위 비트와 더해줌으로써 carry를 0 또는 1로 만들 수 있다.

즉, τ 로 $y_1 y_0$ 에 의한 carry c' 을 보정하기 위하여

$$c''(y) := c'((B')^{-1}(y)) \boxplus \tau(y)$$

을 정의할 수 있다.

Recovery of the Feistel affine maps

$c''(y)$ 는 y_0, y_1 에 대한 2차식으로 실험적 선형 구조에 따라

$$c''(y) = \alpha(y) \cdot \beta(y) \oplus \gamma(y)$$

로 분해된다. 여기서 α, β 는 y_0, y_1 에 대응되는 함수이며, γ 는 affine map들에 의해 보정되어야 하는 1차식이다. 이때, α, β, γ 는 모두 affine map이다.

γ 는 Proposition 6에 의해 제거 가능하다.

Proposition 6. Let $z = x \boxplus y$ be an n -bit addition. Then, $\neg z = \neg x \boxplus \neg y \boxplus 1$.

Proof) n -bit 보수 표현에 대하여

$$\neg x = -x - 1 \pmod{2^n}, \quad \neg y = -y - 1 \pmod{2^n}$$

이다.

$$\begin{aligned} \neg x \boxplus \neg y \boxplus 1 &\equiv (-x - 1) + (-y - 1) + 1 \equiv -(x + y) - 1 \\ &\equiv -z - 1 \equiv \neg z \pmod{2^n} \end{aligned}$$

이므로 $\neg z = \neg x \boxplus \neg y \boxplus 1$ 이다. □

Recovery of the Feistel affine maps

보수 표현의 특성상 y 에 대한 임의의 함수 f 가 주어질 때, $\neg$ 은 모든 입력과 출력에 $f(y) \cdot (1, \dots, 1)$ 을 더하는 것으로 대체할 수 있어 임의의 x 에 대하여

$$x \mapsto x \oplus f(y) \cdot (1, \dots, 1) = \begin{cases} \neg x, & f(y) = 1, \\ x, & f(y) = 0 \end{cases}$$

이다. 이를 Proposition 6에 적용하면

$$(x \boxplus y) \mapsto (x \boxplus y) \oplus \tau(y) \cdot (1, \dots, 1) = \begin{cases} \neg x \boxplus \neg y \boxplus 1, & \tau(y) = 1, \\ x \boxplus y, & \tau(y) = 0 \end{cases}$$

가 된다. 즉,

$$(x \boxplus y) \oplus \tau(y) \cdot (1, \dots, 1) = (x \oplus \tau(y) \cdot (1, \dots, 1)) \boxplus (y \oplus \tau(y) \cdot (1, \dots, 1)) \boxplus \tau(y)$$

으로 정리하여 표현할 수 있다.

Proposition 6는 $\tau(y) = 1$ 일 때 $\neg z \boxplus 1 = \neg x \boxplus \neg y = x' \boxplus y'$ 으로 $\boxplus 1$ 을 걸러내는 것으로 볼 수 있다.

Recovery of the Feistel affine maps

이를 공격 대상인 $\mathbb{O}''$ 에 적용하기 위하여 Proposition 7을 사용한다.

Proposition 7. Consider the setting from the text above. Let $\sigma(x, y)$ be an affine map such that

$$\sigma_i(x, y) = \begin{cases} \beta(y), & i = 0, \\ y_{2+i-1} \oplus \gamma(y), & 1 \leq i \leq n-1, \\ \alpha(y) \oplus \beta(y), & i = n, \\ x_i \oplus \gamma(y), & n+1 < i < N. \end{cases}$$

If it is invertible, then, $\sigma \circ \mathbb{O}'' \circ \sigma^{-1}$ implements $(x, y) \mapsto (x \boxplus y, y)$ for $x, y \in \mathbb{F}_2^n$.

Proof) 먼저 $C_\gamma(x) := x \oplus \gamma(y) \cdot (1, \dots, 1)$ 이라고 하자.

그럼 Proposition 6에 의해 $C_\gamma(u \boxplus v) = C_\gamma(u) \boxplus C_\gamma(v) \boxplus \gamma(y)$ 이다.

또한, $\mathbb{O}''(x, y) = (x \boxplus y_{[2:n+1]} \boxplus c''(y), y)$ 이고 $c''(y) = \alpha(y)\beta(y) \oplus \gamma(y)$ 이므로,

$\mathbb{O}''(x, y) = (x \boxplus y_{[2:n+1]} \boxplus \alpha(y)\beta(y) \oplus \gamma(y), y)$ 이다. 따라서

$$\begin{aligned} ((C_{\gamma(y)}, I) \circ \mathbb{O}'')(x, y) &= (C_{\gamma(y)}, I) (x \boxplus y_{[2:n+1]} \boxplus \alpha(y)\beta(y) \oplus \gamma(y), y) \\ &= (C_{\gamma(y)}(x) \boxplus C_{\gamma(y)}(y_{[2:n+1]}) \boxplus \alpha(y)\beta(y), y) \end{aligned}$$

이다.

Recovery of the Feistel affine maps

한편, 단계 5를 시작하며 적용한 index 변환을 고려하면

$$\sigma_n(x, y)\sigma_0(x, y) \oplus \sigma_0(x, y) = (\alpha(y) \oplus \beta(y))\beta(y) \oplus \beta(y) = \alpha(y)\beta(y)$$

로 $\sigma(x, y)$ 를 Proposition 7에서와 같이 정의하면,

$$(\sigma \circ \mathbb{O}'')(x, y) = (\sigma_{[n+1:2n]}(x, y) \boxplus \sigma_{[0:n]}(x, y) \boxplus \sigma_n(x, y)\sigma_0(x, y) \oplus \sigma_0(x, y), \sigma_{[0:n+1]}(x, y))$$

이다. 따라서

$$(\sigma \circ \mathbb{O}'' \circ \sigma^{-1})(x, y) = (x \boxplus y_{[2:n+1]} \boxplus y_0 y_n, y) = (x \boxplus y, y)$$

로 모듈러 덧셈을 수행하는 Feistel 구조를 완전히 복구하게 된다. σ 가 가역이 아닌 경우 B', τ 등을 다시 선택해야 함

감사합니다



Q&A

주 참고 문헌 : [1]

- [1] Alex Biryukov, Baptiste Lambin, and Aleksei Udovenko.
Cryptanalysis of arx-based white-box implementations.
IACR Transactions on Cryptographic Hardware and Embedded Systems, 2023(3):97–135, 2023.
- [2] Helger Lipmaa and Shiho Moriai.
Efficient algorithms for computing differential properties of addition.
In *International Workshop on Fast Software Encryption*, pages 336–350. Springer, 2001.